LABORATORY 05 – Databases and Network Protocols

AUTORS

Andrés Felipe Chavarro Plazas

David Santiago Espinosa Rojas

TEACHER

Diego Alejandro Murcia Cespedes

COMPUTER NETWORKS

GROUP 01

Bogota, Colombia

March 23, 2025

OBJETIVE

For this laboratory, we are going to analyze DNS, HTTP, and Ethernet frames using Wireshark, to understand how they work. Then, install PostgreSQL on Linux Slackware and SQL Server on Windows Server, creating and managing related databases. Explore Azure services, scaling, and security differences. Configure databases on Linux NetBSD, enable auto-start, connect remotely with DBeaver, and present results.

MATERIALS & TOOLS

- Laboratory computers with internet access.
- Virtualization software.
- Azure cloud services.
- Database manager

INTRODUCTION

In this lab, we will focus on the configuration and analysis of databases and network protocols. We will use Wireshark to capture and examine DNS, HTTP, and Ethernet frame messages, identifying and explaining their fields.

In addition, we will install and configure database management systems such as PostgreSQL on a Linux Slackware virtual machine and SQL Server on a Windows Server virtual machine, creating custom databases and managing users with specific permissions. We will also explore Microsoft Azure services, evaluating the advantages of a cloud infrastructure versus an on-premises one, analyzing service models (IaaS, PaaS, and SaaS), the differences between vertical and horizontal scaling, and the use of TLS for data transmission security.

Finally, we will configure databases to start automatically with the operating system and establish remote connections using DBeaver (or any other similar tool) to verify the contents of the created tables.

THEORETICAL FRAMEWORK

Wireshark is a network analysis tool used to capture and examine network traffic in real-time. It helps identify connectivity and security issues by analyzing protocols like DNS or HTTP, providing a clear view of requests and responses. This makes it easier to troubleshoot problems and understand network behavior, like the headers from the packages build in the transport layer by protocols like UDP and TCP.

On the other hand, a Database Management System allows for the efficient creation, management, and manipulation of databases. DBMSs ensure data integrity, provide secure access through user permissions, and simplify backup and recovery processes. So, there are many database engines for different systems and applications.

PostgreSQL is an open-source relational database system known for its reliability and ability to handle complex data structures, including formats like JSON and XML. It offers advanced features such as foreign keys, triggers, and stored procedures, making it a powerful tool for managing large and complex datasets.

SQL Server, is widely used in enterprise environments due to its seamless integration with other Microsoft products. It supports ACID (Atomicity, Consistency, Isolation, and Durability) transactions, replication, and data partitioning, ensuring data integrity and high availability.

Cloud computing enables access to computing resources over the internet, providing flexibility and scalability. Microsoft Azure offers services under three main models: IaaS for virtualized infrastructure, PaaS for application development and deployment, and SaaS for ready-to-use applications. This allows organizations to scale resources based on demand and reduce operational costs.

Scalability allows a system to handle an increased workload without affecting performance. Vertical scalability involves increasing the resources of a single server, while horizontal scalability adds more servers to distribute the load, which is often more effective for large-scale systems.

TLS encrypts data transmitted between clients and servers, ensuring privacy and data integrity. This is particularly important for cloud-based databases like Azure SQL Database, where secure data transmission is essential.

Setting up databases to start automatically using systemd on Linux or Windows Services helps ensure that they are available immediately after a system reboot. Tools like DBeaver allow remote access and management of databases using secure protocols like TCP/IP or SSL/TLS, providing a convenient way to monitor and update data.

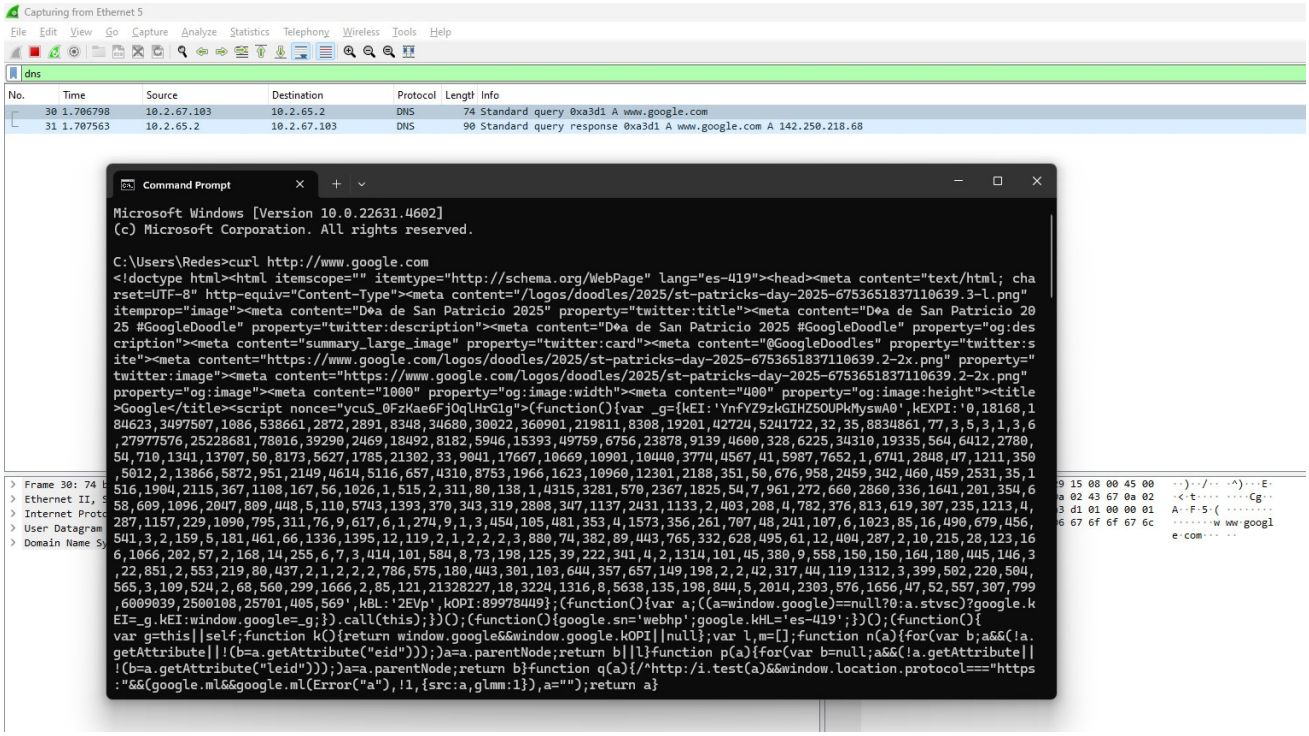
REVIEW OF NETWORK PROTOCOLS

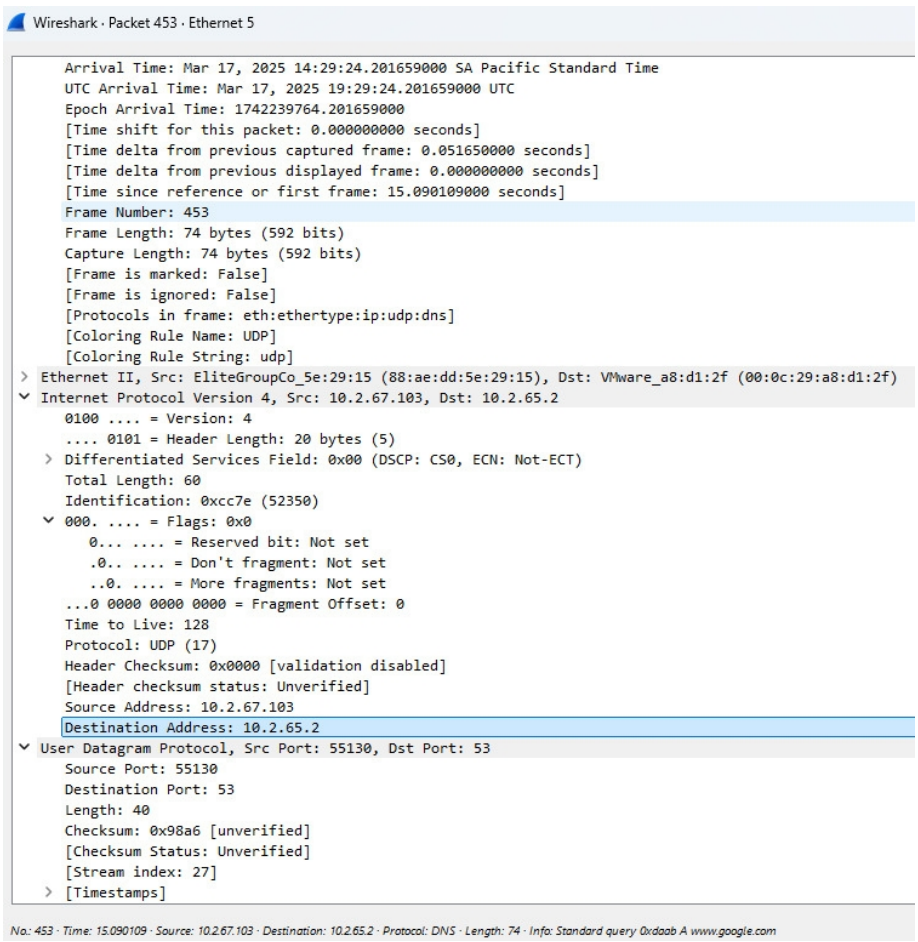
In this section, we review and analyze some application layer protocols. As with the previous labs, we will use the Wireshark tool to analyze the headers of packets received from the IP protocol at the transport and network layers.

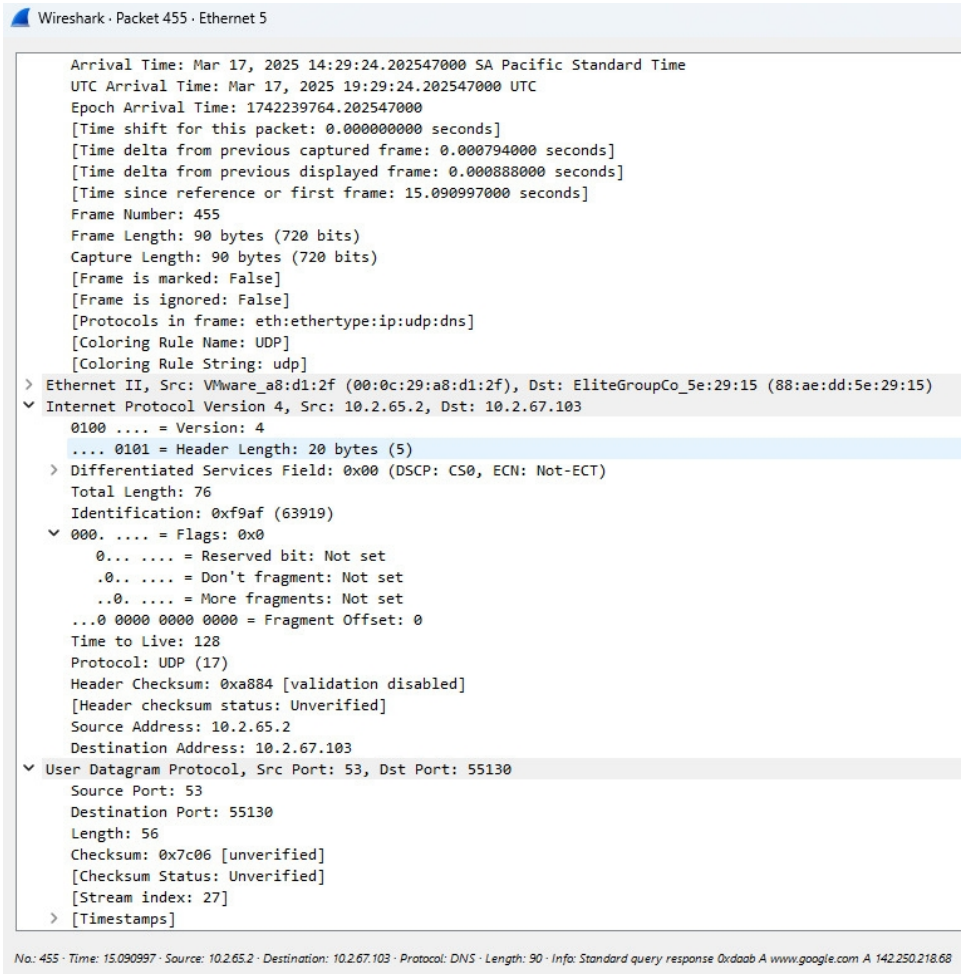
DNS MESSAGES

For the first query, we are going to filter the packets received from the DNS. Then, analyze their content and identify their components to verify the elements obtained from the different network layers.

To begin, we run the `***dns***` query in Wireshark, then we have to do the query requested page with the `***curl http://www.google.com***` command from the console. This will retrieve the packets for that page as the image shows.


Capturing from Ethernet 5
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
dns
No. Time Source Destination Protocol Length Info
30 1.706798 10.2.67.103 10.2.65.2 DNS 74 Standard query 0xa3d1 A www.google.com
31 1.707563 10.2.65.2 10.2.67.103 DNS 90 Standard query response 0xa3d1 A www.google.com A 142.250.218.68
Command Prompt
Microsoft Windows [Version 10.0.22631.4602]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Redes>curl http://www.google.com
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="es-419"><head><meta content="text/html; charset=UTF-8" http-equiv="Content-Type"><meta content="/logos/doodles/2025/st-patricks-day-2025-6753651837110639.3-l.png" itemprop="image"><meta content="D♦a de San Patricio 2025" property="twitter:title"><meta content="D♦a de San Patricio 2025 #GoogleDoodle" property="twitter:description"><meta content="D♦a de San Patricio 2025 #GoogleDoodle" property="og:description"><meta content="summary_large_image" property="twitter:card"><meta content="@GoogleDoodles" property="twitter:site"><meta content="https://www.google.com/logos/doodles/2025/st-patricks-day-2025-6753651837110639.2-2x.png" property="twitter:image"><meta content="https://www.google.com/logos/doodles/2025/st-patricks-day-2025-6753651837110639.2-2x.png" property="og:image"><meta content="1000" property="og:image:width"><meta content="400" property="og:image:height"><title>Google</title><script nonce="ycuS_0FzKae6FjOqlHrG1g">(function(){var _g={kEI:'YnfYZ9zkGIHZ5OUPkMyswA0',kEXPI:'0,18168,184623,3497507,1086,538661,2872,2891,8348,34680,30022,360901,219811,8308,19201,42724,5241722,32,35,8834861,77,3,5,3,1,3,6,27977576,25228681,78016,39290,2469,18492,8182,5946,15393,49759,6756,23878,9139,4600,328,6225,34310,19335,564,6412,2780,54,710,1341,13707,50,8173,5627,1785,21302,33,9041,17667,10669,10901,10440,3774,4567,41,5987,7652,1,6741,2848,47,1211,350,5012,2,13866,5872,951,2149,4614,5116,657,4310,8753,1966,1623,10960,12301,2188,351,50,676,958,2459,342,460,459,2531,35,1516,1904,2115,367,1108,167,56,1026,1,515,2,311,80,138,1,4315,3281,570,2367,1825,54,7,961,272,660,2860,336,1641,201,354,658,609,1096,2047,809,448,5,110,5743,1393,370,343,319,2808,347,1137,2431,1133,2,403,208,4,782,376,813,619,307,235,1213,4,287,1157,229,1090,795,311,76,9,617,6,1,274,9,1,3,454,105,481,353,4,1573,356,261,707,48,241,107,6,1023,85,16,490,679,456,541,3,2,159,5,181,461,66,1336,1395,12,119,2,1,2,2,2,3,880,74,382,89,443,765,332,628,495,61,12,404,287,2,10,215,28,123,166,1066,202,57,2,168,14,255,6,7,3,414,101,584,8,73,198,125,39,222,341,4,2,1314,101,45,380,9,558,150,150,164,180,445,146,3,22,851,2,553,219,80,437,2,1,2,2,2,786,575,180,443,301,103,644,357,657,149,198,2,2,42,317,44,119,1312,3,399,502,220,504,565,3,109,524,2,68,560,299,1666,2,85,121,21328227,18,3224,1316,8,5638,135,198,844,5,2014,2303,576,1656,47,52,557,307,799,6009039,2500108,25701,405,569',kBL:'2EVp',kOPI:89978449};(function(){var a;((a=window.google)==null?0:a.stvsc)?google.kEI=_g.kEI:window.google=_g;}).call(this);})();(function(){google.sn='webhp';google.kHL='es-419';})();(function(){var g=this||self;function k(){return window.google&&window.google.kOPI||null};var l,m=[];function n(a){for(var b;a&&(!a.getAttribute||!(b=a.getAttribute("eid")));)a=a.parentNode;return b||l}function p(a){for(var b=null;a&&(!a.getAttribute||!(b=a.getAttribute("leid")));)a=a.parentNode;return b}function q(a){/^http:/i.test(a)&&window.location.protocol==="https:"&&(google.ml&&google.ml(Error("a"),!1,{src:a,glmm:1}),a="");return a}


Wireshark · Packet 453 · Ethernet 5
Arrival Time: Mar 17, 2025 14:29:24.201659000 SA Pacific Standard Time
UTC Arrival Time: Mar 17, 2025 19:29:24.201659000 UTC
Epoch Arrival Time: 1742239764.201659000
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 0.051650000 seconds]
[Time delta from previous displayed frame: 0.000000000 seconds]
[Time since reference or first frame: 15.090109000 seconds]
Frame Number: 453
Frame Length: 74 bytes (592 bits)
Capture Length: 74 bytes (592 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:udp:dns]
[Coloring Rule Name: UDP]
[Coloring Rule String: udp]
Ethernet II, Src: EliteGroupCo_5e:29:15 (88:ae:dd:5e:29:15), Dst: VMware_a8:d1:2f (00:0c:29:a8:d1:2f)
Internet Protocol Version 4, Src: 10.2.67.103, Dst: 10.2.65.2
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 60
Identification: 0xcc7e (52350)
000. .... = Flags: 0x0
0... .... = Reserved bit: Not set
.0.. .... = Don't fragment: Not set
..0. .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: UDP (17)
Header Checksum: 0x0000 [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.2.67.103
Destination Address: 10.2.65.2
User Datagram Protocol, Src Port: 55130, Dst Port: 53
Source Port: 55130
Destination Port: 53
Length: 40
Checksum: 0x98a6 [unverified]
[Checksum Status: Unverified]
[Stream index: 27]
[Timestamps]
No.: 453 · Time: 15.090109 · Source: 10.2.67.103 · Destination: 10.2.65.2 · Protocol: DNS · Length: 74 · Info: Standard query 0xdaab A www.google.com



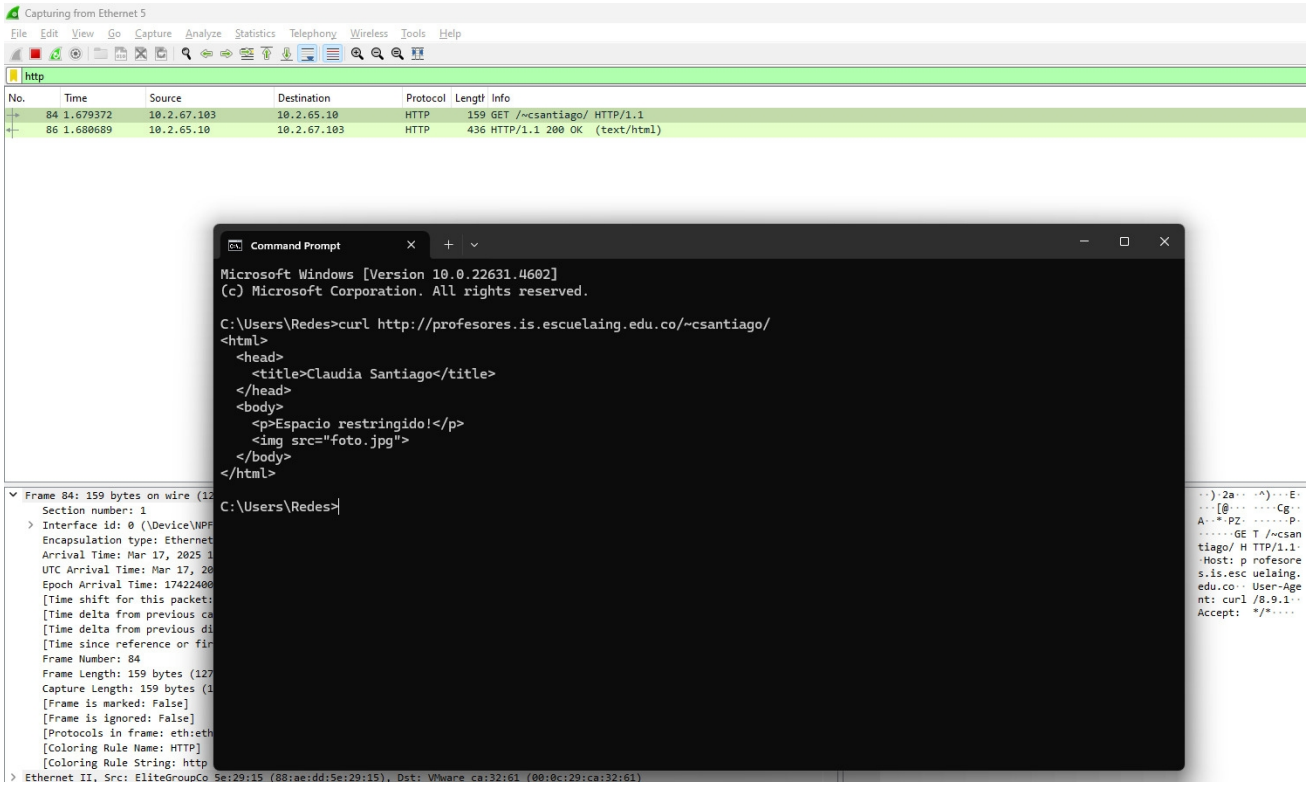
The screenshot shows a standard DNS query for the domain www.google.com. The request was sent from IP address 10.2.67.103 to the DNS server at 10.2.65.2 over port 53. The query uses the UDP protocol because it is faster and more efficient for low-overhead requests like DNS. The Source Port is 55130, indicating that the query was initiated from a dynamic port on the client. The Transaction ID of the query is 0xc0c7, allowing the client to relate the response to this specific request. The request is encapsulated in an IPv4 packet.

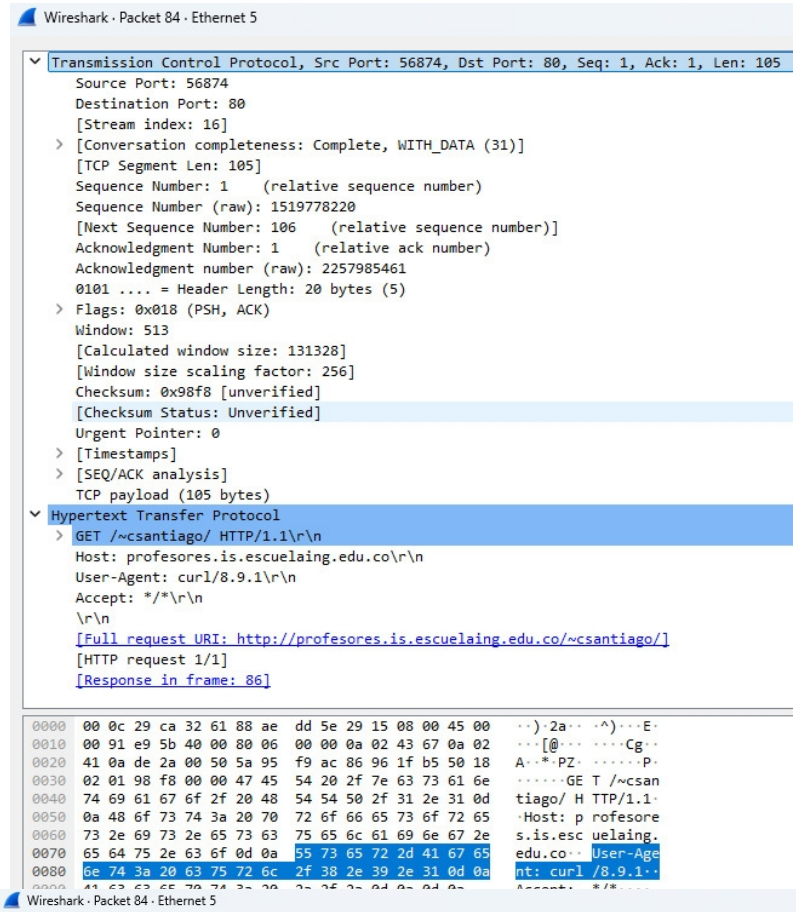
In the case of the response, the client at 10.2.67.103 sent a DNS query to the server 10.2.65.2 on port 53 to resolve www.google.com. The response arrived with the same Transaction ID (0xc0c7), returning the IP 142.250.190.78 and a NOERROR code, indicating success. The Ethernet frame showed the source MAC address 88:ae:dd:5e:29:15 and the destination MAC address 00:0c:29:a8:d1:2f, encapsulated in an IPv4 packet.

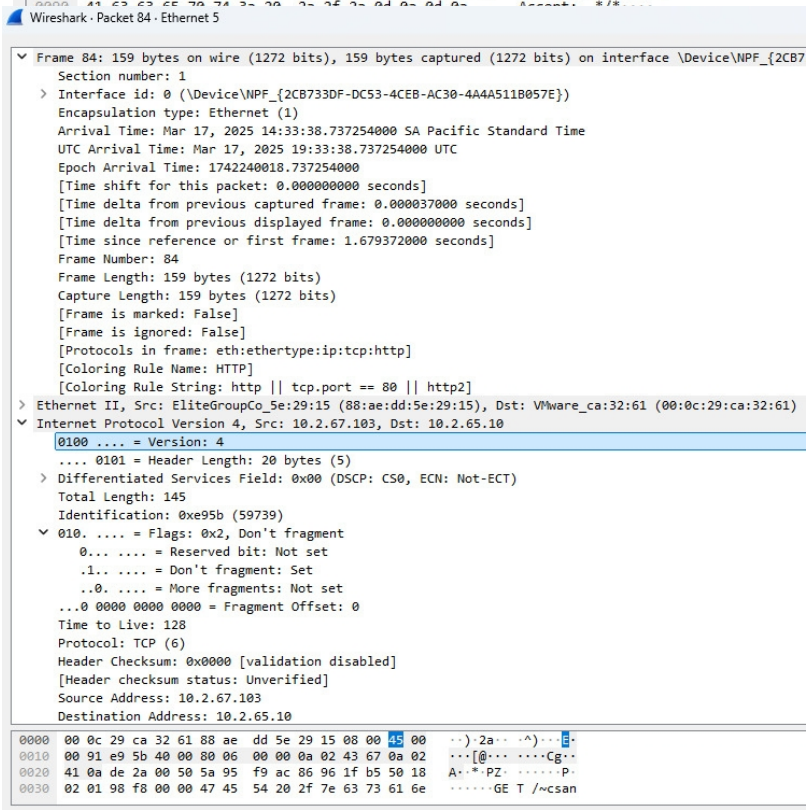
HTTP MESSAGES

In the second query, we are going to review the packets received via the HTTP protocol when accessing the requested page. With these, we'll analyze their components and deduce their nature.

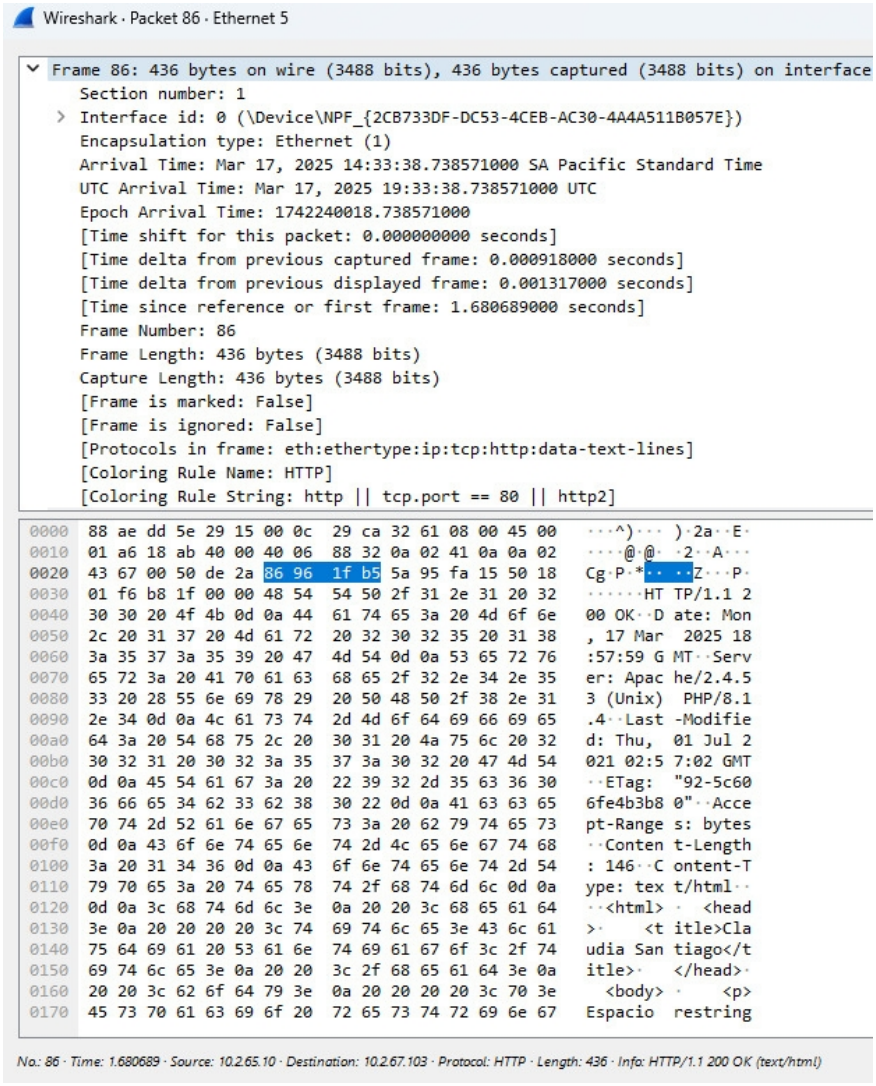
To begin, we are going to run an `***http***` query with Wireshark, which allows us to view all the packets received from this protocol using the command `***curl http://profesores.is.escuelaing.edu.co/~csantiago/***` in the console. With these packets, we can verify the components that comprise it and which ones originate from other layers of the network.

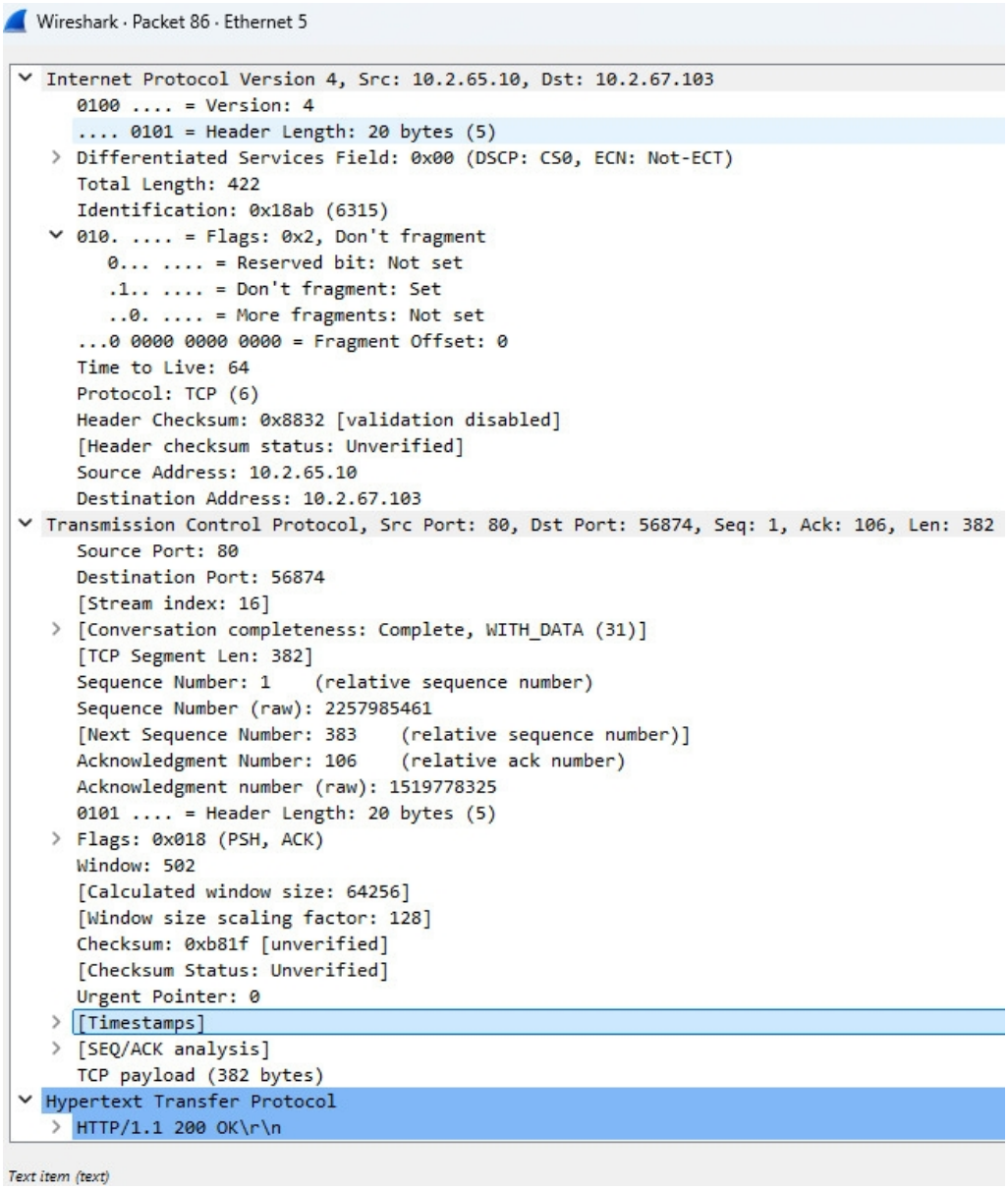



Wireshark · Packet 84 · Ethernet 5
Transmission Control Protocol, Src Port: 56874, Dst Port: 80, Seq: 1, Ack: 1, Len: 105
Source Port: 56874
Destination Port: 80
[Stream index: 16]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 105]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 1519778220
[Next Sequence Number: 106 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 2257985461
0101 .... = Header Length: 20 bytes (5)
Flags: 0x018 (PSH, ACK)
Window: 513
[Calculated window size: 131328]
[Window size scaling factor: 256]
Checksum: 0x98f8 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SEQ/ACK analysis]
TCP payload (105 bytes)
Hypertext Transfer Protocol
GET /~csantiago/ HTTP/1.1\r\n
Host: profesores.is.escuelaing.edu.co\r\n
User-Agent: curl/8.9.1\r\n
Accept: */*\r\n
\r\n
[Full request URI: http://profesores.is.escuelaing.edu.co/~csantiago/]
[HTTP request 1/1]
[Response in frame: 86]
0000 00 0c 29 ca 32 61 88 ae dd 5e 29 15 08 00 45 00
0010 00 91 e9 5b 40 00 80 06 00 00 0a 02 43 67 0a 02
0020 41 0a de 2a 00 50 5a 95 f9 ac 86 96 1f b5 50 18
0030 02 01 98 f8 00 00 47 45 54 20 2f 7e 63 73 61 6e
0040 74 69 61 67 6f 2f 20 48 54 54 50 2f 31 2e 31 0d
0050 0a 48 6f 73 74 3a 20 70 72 6f 66 65 73 6f 72 65
0060 73 2e 69 73 2e 65 73 63 75 65 6c 61 69 6e 67 2e
0070 65 64 75 2e 63 6f 0d 0a 55 73 65 72 2d 41 67 65
0080 6e 74 3a 20 63 75 72 6c 2f 38 2e 39 2e 31 0d 0a


Wireshark · Packet 84 · Ethernet 5
Frame 84: 159 bytes on wire (1272 bits), 159 bytes captured (1272 bits) on interface \Device\NPF_{2CB7
Section number: 1
Interface id: 0 (\Device\NPF_{2CB733DF-DC53-4CEB-AC30-4A4A511B057E})
Encapsulation type: Ethernet (1)
Arrival Time: Mar 17, 2025 14:33:38.737254000 SA Pacific Standard Time
UTC Arrival Time: Mar 17, 2025 19:33:38.737254000 UTC
Epoch Arrival Time: 1742240018.737254000
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 0.000037000 seconds]
[Time delta from previous displayed frame: 0.000000000 seconds]
[Time since reference or first frame: 1.679372000 seconds]
Frame Number: 84
Frame Length: 159 bytes (1272 bits)
Capture Length: 159 bytes (1272 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp:http]
[Coloring Rule Name: HTTP]
[Coloring Rule String: http || tcp.port == 80 || http2]
Ethernet II, Src: EliteGroupCo_5e:29:15 (88:ae:dd:5e:29:15), Dst: VMware_ca:32:61 (00:0c:29:ca:32:61)
Internet Protocol Version 4, Src: 10.2.67.103, Dst: 10.2.65.10
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 145
Identification: 0xe95b (59739)
010. .... = Flags: 0x2, Don't fragment
0... .... = Reserved bit: Not set
.1.. .... = Don't fragment: Set
..0. .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: TCP (6)
Header Checksum: 0x0000 [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.2.67.103
Destination Address: 10.2.65.10
0000 00 0c 29 ca 32 61 88 ae dd 5e 29 15 08 00 45 00
0010 00 91 e9 5b 40 00 80 06 00 00 0a 02 43 67 0a 02
0020 41 0a de 2a 00 50 5a 95 f9 ac 86 96 1f b5 50 18
0030 02 01 98 f8 00 00 47 45 54 20 2f 7e 63 73 61 6e

The client at 10.2.67.103 sent an HTTP GET request to the server 10.2.65.10 on port 80 to access the resource /~csantiago/. The request includes the Host: profesores.is.escuelaing.edu.co header and the curl/8.9.1 user agent. The Ethernet frame shows the source MAC address 88:ae:dd:5e:29:15 and the destination MAC address 00:0c:29:ca:32:61, encapsulated in an IPv4 packet and transmitted using TCP on port 80.



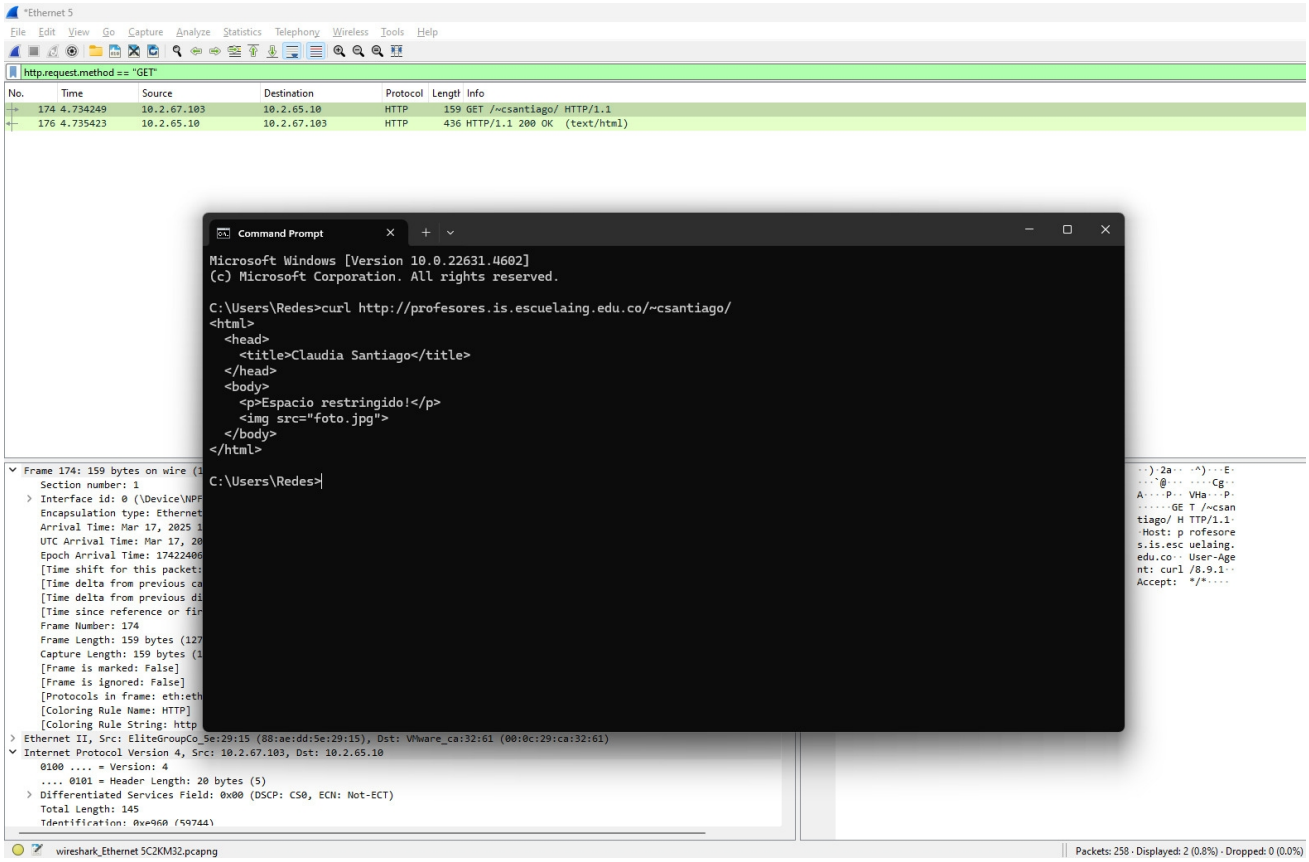


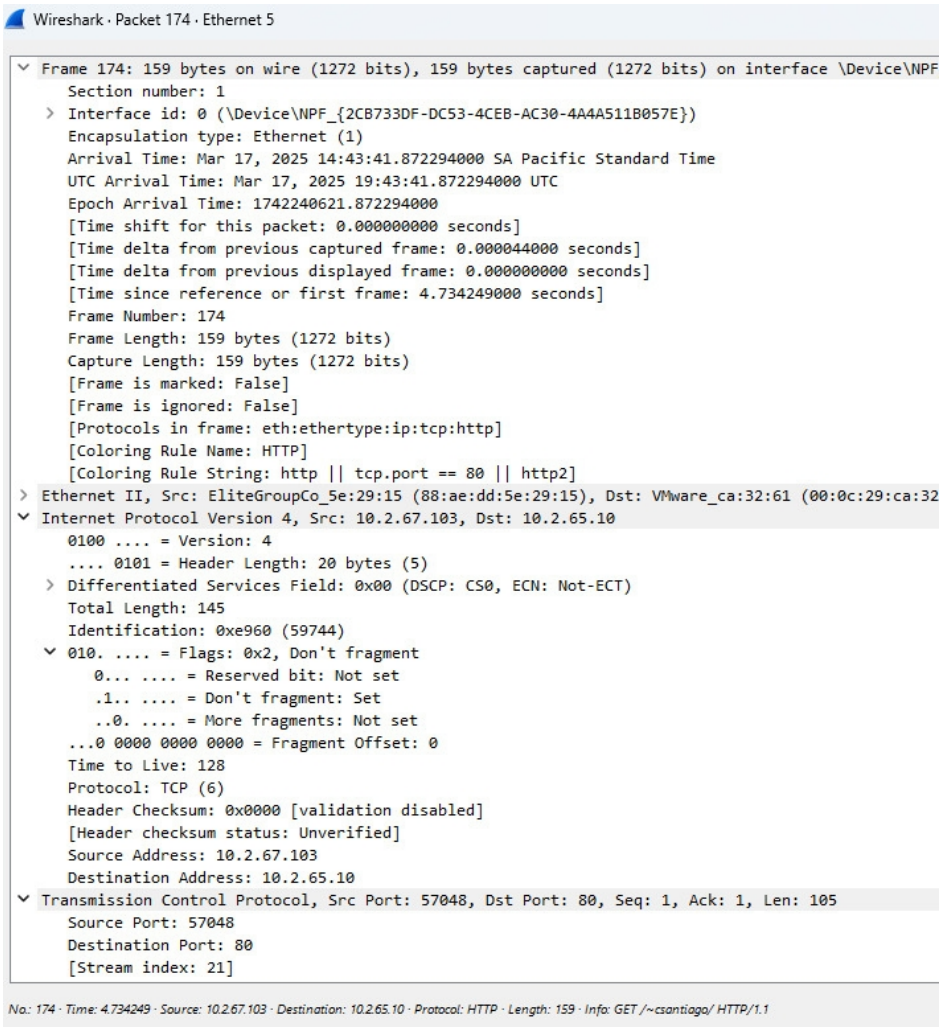
The HTTP response indicates that the GET request was successful, returning a 200 OK code. The Apache/2.4.53 (Unix) server running PHP/8.1.7 responded on March 17, 2025 at 18:57:59 GMT and sent a 146-byte HTML file with type text/html. The file includes a simple HTML page with the title "Claudia Santiago" and a message in the body stating "Restricted space", suggesting that access to the resource might be limited or protected.

ETHERNET FRAMES

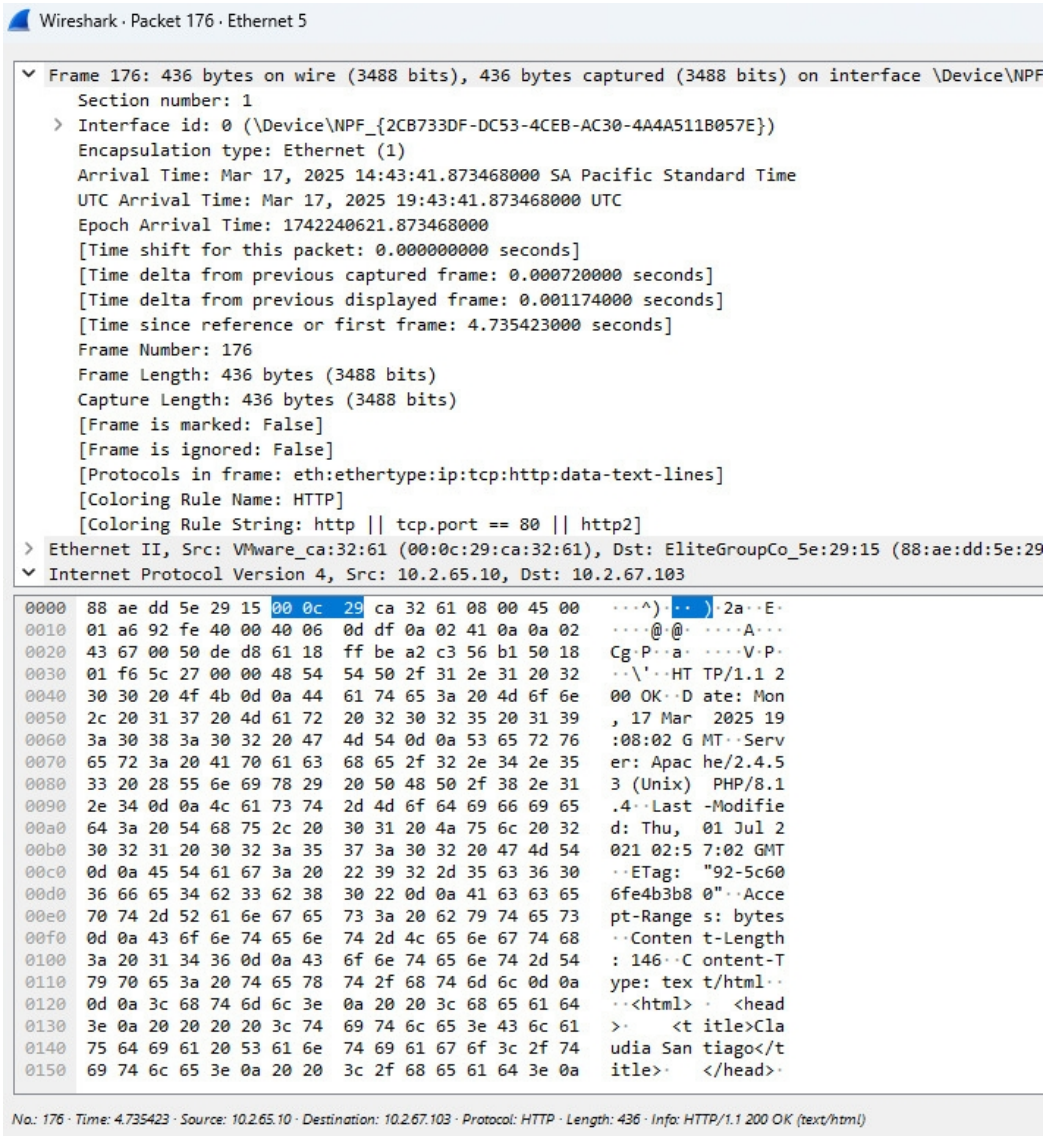
In this last query, we'll make another HTTP request focused on the GET method on the requested page on the guide. This, allows us to obtain the request and response and analyze them after that with the specialist software.

Finally, we'll use Wireshark with the `***http.request.method == "GET"***` query, which will give us the target packets when we make the console request to the page with the `***curl http://profesores.is.escuelaing.edu.co/~csantiago/***` command. This way, we can get a response to the given request.





The HTTP request was transmitted via an Ethernet frame from source MAC address 88:ae:dd:5e:29:15 to destination MAC address 00:0c:29:ca:32:61. The Ethernet frame is encapsulated in Ethernet II format, with a protocol type of 0x0800 indicating that the frame content is an IPv4 packet. The total frame length is 159 bytes, which includes the 14-byte Ethernet header, followed by the IP packet and the TCP segment containing the HTTP request. The frame was sent from source port 57048 to destination port 80, confirming that it is a standard HTTP request to a web server.



The HTTP response was transmitted via an Ethernet frame from source MAC address 00:0c:29:ca:32:61 to destination MAC address 88:ae:dd:5e:29:15. The Ethernet frame is encapsulated in Ethernet II format, with protocol type 0x0800 indicating that it is carrying an IPv4 packet. The total frame length is 436 bytes, which includes the 14-byte Ethernet header, the IP packet, the TCP segment, and the HTTP message body. This response was sent from port 80 to port 57048, confirming the successful delivery of the HTTP response to the client.

BASE SOFTWARE INSTALLATION

In this section, we are going to use databases to see how we can use different virtualized operating systems in VMware for deployment and connection. Cloud services will also be used to see how the databases and their managers work in a more professional environment, that's the objective.

POSTGRESQL – SLACKWARE

For this first database, we'll install the Postgres DBMS on our virtual machine running Slackware. To do this, we need to download the software binaries from the Postgres download page (https://www.postgresql.org/ftp/source/v15.12/). Then, we use the `wget` command and start the download from our root user.

```
root@root:~# wget https://ftp.postgresql.org/pub/source/v15.12/postgresql-15.12.tar.gz
--2025-03-17 17:18:10--  https://ftp.postgresql.org/pub/source/v15.12/postgresql-15.12.tar.gz
Resolving ftp.postgresql.org (ftp.postgresql.org)... 147.75.85.69, 72.32.157.246, 87.238.57.227,
 ...
Connecting to ftp.postgresql.org (ftp.postgresql.org)|147.75.85.69|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 30477320 (29M) [application/octet-stream]
Saving to: ‘postgresql-15.12.tar.gz.1’

postgresql-15.12.tar.gz 100%[===============================>]  29.07M  1.58MB/s    in 19s

2025-03-17 17:18:30 (1.55 MB/s) - ‘postgresql-15.12.tar.gz.1’ saved [30477320/30477320]
```

This download provides a compressed file with the `***.tar***` extension, typical of Unix binary packages. To decompress it, use the command `***tar -xfvz postgresx.tar***`, which will generate the folder with the files needed to continue installing the program.

Now we are going to configure a postgres user, starting with creating a user and assigning the necessary permissions so that it can run on the system, so we use the `***useradd -m -U -r -d /var/lib/postgresql -s /bin/bash postgres***` command for its creation and grant him with full permissions as sudo user. Now, let's create a folder where the Postgres user will be able to run the database engine. So, we create a directory like `***mkdir -p /usr/local/pgsql/data***`, and we'll change the folder's owner with the `chown` command to the user we created.

```
root@root:~/postgresql-15.12# mkdir -p /usr/local/pgsql/data
root@root:~/postgresql-15.12# chown postgres /usr/local/pgsql/data
root@root:~/postgresql-15.12# su postgres
postgres@root:/root/postgresql-15.12$
```

With the directory ready, we can run a configuration script located in the same directory where we downloaded the Postgres binary. To do this, we run the command `***./configure –prefix=/usr/local/pgsql***`, which will generate the binary executable for that user.

```
root@root:~/postgresql-15.12# ./configure --prefix=/usr/local/pgsql
```

Finally, we have to package the application using the make tool, which converts the binaries into executables for users. So, we simply issue the `***make***` command and watch as the application takes a while to compile. Next, we are going to use the `make install` command to install the application as an executable from its own command or desktop image. Both processes look like this:

```
make[4]: Entering directory '/root/postgresql-15.12/src/port'
make[4]: Nothing to be done for 'all'.
make[4]: Leaving directory '/root/postgresql-15.12/src/port'
make -C ../../../src/common all
make[4]: Entering directory '/root/postgresql-15.12/src/common'
make[4]: Nothing to be done for 'all'.
make[4]: Leaving directory '/root/postgresql-15.12/src/common'
make[3]: Leaving directory '/root/postgresql-15.12/src/test/regress'
rm -f pg_regress.o && ln -s ../../../src/test/regress/pg_regress.o .
gcc -Wall -Wmissing-prototypes -Wpointer-arith -Wdeclaration-after-statement -Werror=vla -Wendif
-labels -Wmissing-format-attribute -Wimplicit-fallthrough=3 -Wcast-function-type -Wformat-securi
ty -fno-strict-aliasing -fwrapv -fexcess-precision=standard -Wno-format-truncation -Wno-stringop
-truncation -O2 isolation_main.o pg_regress.o -L../../../src/port -L../../../src/common   -Wl,--
as-needed -Wl,-rpath,'/usr/local/pgsql/lib',--enable-new-dtags  -lpgcommon -lpgport -lz -lreadli
ne -lpthread -lrt -ldl -lm  -o pg_isolation_regress
make[2]: Leaving directory '/root/postgresql-15.12/src/test/isolation'
make -C test/perl all
make[2]: Entering directory '/root/postgresql-15.12/src/test/perl'
make[2]: Nothing to be done for 'all'.
make[2]: Leaving directory '/root/postgresql-15.12/src/test/perl'
make[1]: Leaving directory '/root/postgresql-15.12/src'
make -C config all
make[1]: Entering directory '/root/postgresql-15.12/config'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/root/postgresql-15.12/config'
```

```
make -C ../../../src/common all
make[3]: Entering directory '/root/postgresql-15.12/src/common'
make[3]: Nothing to be done for 'all'.
make[3]: Leaving directory '/root/postgresql-15.12/src/common'
/bin/mkdir -p '/usr/local/pgsql/lib/pgxs/src/test/isolation'
/bin/ginstall -c  pg_isolation_regress '/usr/local/pgsql/lib/pgxs/src/test/isolation/pg_isolatio
n_regress'
/bin/ginstall -c  isolationtester '/usr/local/pgsql/lib/pgxs/src/test/isolation/isolationtester'
make[2]: Leaving directory '/root/postgresql-15.12/src/test/isolation'
make -C test/perl install
make[2]: Entering directory '/root/postgresql-15.12/src/test/perl'
make[2]: Nothing to be done for 'install'.
make[2]: Leaving directory '/root/postgresql-15.12/src/test/perl'
/bin/mkdir -p '/usr/local/pgsql/lib/pgxs/src'
/bin/ginstall -c -m 644 Makefile.global '/usr/local/pgsql/lib/pgxs/src/Makefile.global'
/bin/ginstall -c -m 644 Makefile.port '/usr/local/pgsql/lib/pgxs/src/Makefile.port'
/bin/ginstall -c -m 644 ./Makefile.shlib '/usr/local/pgsql/lib/pgxs/src/Makefile.shlib'
/bin/ginstall -c -m 644 ./nls-global.mk '/usr/local/pgsql/lib/pgxs/src/nls-global.mk'
make[1]: Leaving directory '/root/postgresql-15.12/src'
make -C config install
make[1]: Entering directory '/root/postgresql-15.12/config'
/bin/mkdir -p '/usr/local/pgsql/lib/pgxs/config'
/bin/ginstall -c -m 755 ./install-sh '/usr/local/pgsql/lib/pgxs/config/install-sh'
/bin/ginstall -c -m 755 ./missing '/usr/local/pgsql/lib/pgxs/config/missing'
make[1]: Leaving directory '/root/postgresql-15.12/config'
```

Now that we have an application running, let's initialize the Postgres services. In this case, we run the commands `***/usr/local/pgsql/bin/initdb -D /usr/local/pgsql/data***` and `***/usr/local/pgsql/bin/pg_ctl -D /usr/local/pgsql/data start***`, which are used to start the local server where we can store our database. We can manage our information, permissions, users, and more without any problems.

```
  LC_TIME:     en_US.UTF-8
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /usr/local/pgsql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... Etc/GMT+5
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local
 and --auth-host, the next time you run initdb.

Success. You can now start the database server using:

    /usr/local/pgsql/bin/pg_ctl -D /usr/local/pgsql/data -l logfile start
```

```
postgres@root:/root/postgresql-15.12$ /usr/local/pgsql/bin/pg_ctl -D /usr/local/pgsql/data start
could not change directory to "/root/postgresql-15.12": Permission denied
waiting for server to start....2025-03-17 17:30:37.033 -05 [11704] LOG:  starting PostgreSQL 15.
12 on i686-pc-linux-gnu, compiled by gcc (GCC) 11.2.0, 32-bit
2025-03-17 17:30:37.033 -05 [11704] LOG:  listening on IPv6 address "::1", port 5432
2025-03-17 17:30:37.033 -05 [11704] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2025-03-17 17:30:37.036 -05 [11704] LOG:  listening on Unix socket "/tmp/.s.PGSQL.5432"
2025-03-17 17:30:37.037 -05 [11707] LOG:  database system was shut down at 2025-03-17 17:26:24 -
05
2025-03-17 17:30:37.039 -05 [11704] LOG:  database system is ready to accept connections
 done
server started
```

Finally, we'll be able to enter the information requested in the guide, along with the required permissions and users. Therefore, we enter the `***/usr/local/pgsql/bin/createbd*** ` command, which allows us to generate a new database that we'll call "*lab*". Once the database is created, we enter it using the `***/usr/local/pgsql/bin/psql lab***` command. This will allow us to begin executing SQL code in the terminal, which will then be reflected in the database.1

```
postgres@root:~$ /usr/local/pgsql/bin/createdb lab
2025-03-17 17:32:39.073 -05 [11726] ERROR:  database "lab" already exists
2025-03-17 17:32:39.073 -05 [11726] STATEMENT:  CREATE DATABASE lab;
createdb: error: database creation failed: ERROR:  database "lab" already exists
postgres@root:~$
```

```
postgres@root:~$ /usr/local/pgsql/bin/psql lab
psql (15.12)
Type "help" for help.
```

In this case, we were asked to create a database of tourist sites in Colombia. We also needed to create a user for each group member. Therefore, we created the following tables, usernames, and permissions.

- Users:

```
lab=# CREATE USER andres WITH PASSWORD '123321';
CREATE ROLE
lab=# CREATE USER david WITH PASSWORD '123321';
CREATE ROLE
lab=#
```

```
lab=# ALTER USER andres WITH SUPERUSER;
ALTER ROLE
lab=# ALTER USER david WITH SUPERUSER;
ALTER ROLE
lab=#
```

- Tables:

```
lab=# CREATE TABLE Countries (
lab(#     country_id INT PRIMARY KEY,
lab(#     name VARCHAR(100) NOT NULL
lab(# );
2025-03-21 15:33:35.091 -05 [1477] ERROR:  relation "countries" already exists
2025-03-21 15:33:35.091 -05 [1477] STATEMENT:  CREATE TABLE Countries (
            country_id INT PRIMARY KEY,
            name VARCHAR(100) NOT NULL
        );
ERROR:  relation "countries" already exists
lab=#
```

```
lab=# CREATE TABLE Cities (
lab(#     city_id INT PRIMARY KEY,
lab(#     name VARCHAR(100) NOT NULL,
lab(#     country_id INT,
lab(#     FOREIGN KEY (country_id) REFERENCES Countries(country_id)
lab(# );
2025-03-21 15:34:35.113 -05 [1477] ERROR:  relation "cities" already exists
2025-03-21 15:34:35.113 -05 [1477] STATEMENT:  CREATE TABLE Cities (
            city_id INT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            country_id INT,
            FOREIGN KEY (country_id) REFERENCES Countries(country_id)
        );
ERROR:  relation "cities" already exists
lab=#
```

```
lab=# CREATE TABLE TouristSites (
lab(#     site_id INT PRIMARY KEY,
lab(#     name VARCHAR(200) NOT NULL,
lab(#     description TEXT,
lab(#     city_id INT,
lab(#     rating DECIMAL(3, 1),
lab(#     visited BOOLEAN,
lab(#     FOREIGN KEY (city_id) REFERENCES Cities(city_id)
lab(# );
2025-03-21 15:36:56.682 -05 [1550] ERROR:  relation "touristsites" already exists
2025-03-21 15:36:56.682 -05 [1550] STATEMENT:  CREATE TABLE TouristSites (
            site_id INT PRIMARY KEY,
            name VARCHAR(200) NOT NULL,
            description TEXT,
            city_id INT,
            rating DECIMAL(3, 1),
            visited BOOLEAN,
            FOREIGN KEY (city_id) REFERENCES Cities(city_id)
        );
ERROR:  relation "touristsites" already exists
lab=#
```

In this section, we had a some problems inserting the tables, so we saw an error message indicating that the table already exists. Obviously, the tables were created perfectly, as you'll see later.

- Data insert:

```
lab=# INSERT INTO Countries (country_id, name) VALUES (1, 'Colombia');
2025-03-21 15:38:02.103 -05 [1550] ERROR:  duplicate key value violates unique constraint "count
ries_pkey"
2025-03-21 15:38:02.103 -05 [1550] DETAIL:  Key (country_id)=(1) already exists.
2025-03-21 15:38:02.103 -05 [1550] STATEMENT:  INSERT INTO Countries (country_id, name) VALUES (
1, 'Colombia');
ERROR:  duplicate key value violates unique constraint "countries_pkey"
DETAIL:  Key (country_id)=(1) already exists.
lab=#
```

```
lab=# INSERT INTO Cities (city_id, name, country_id) VALUES (1, 'Bogota', 1), (2, 'Cartagena', '
1'), (3, 'Medellin', 1);
INSERT 0 3
lab=#
```

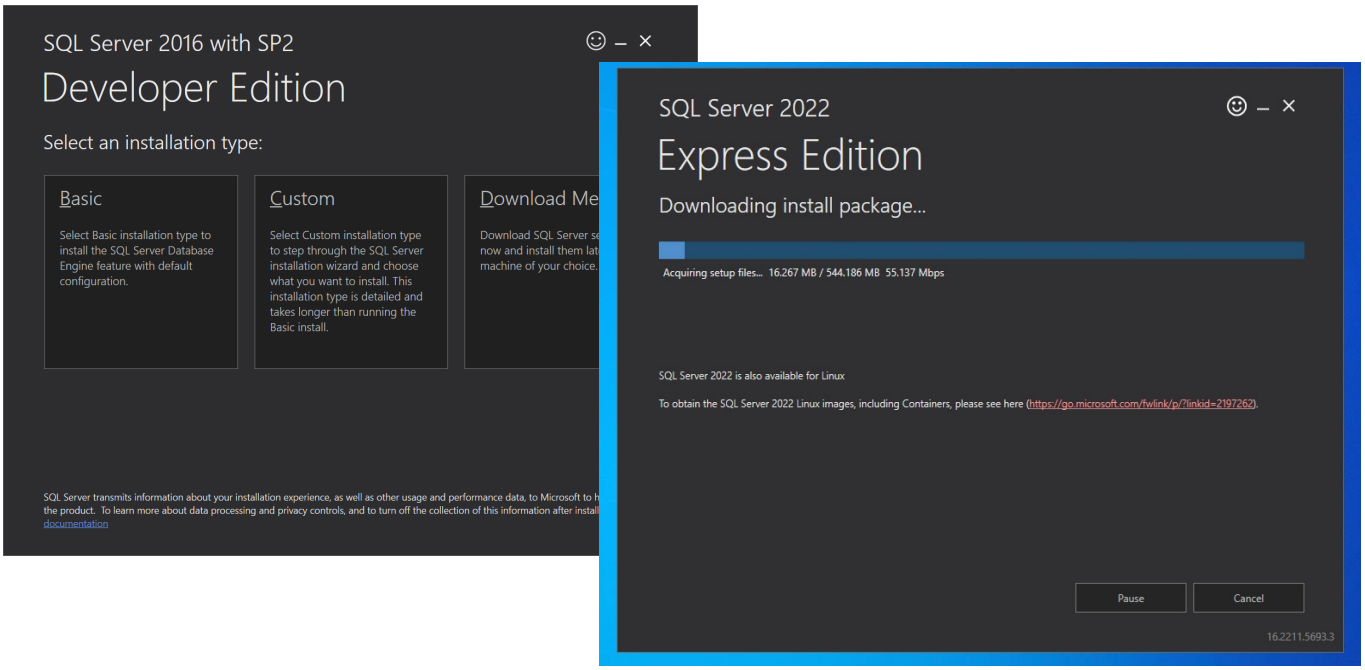
```
lab=# INSERT INTO TouristSites (site_id, name, description, city_id, rating, visited) VALUES (1,
 'Monserrate', 'Iconic monument in Bogota', 1, 4.7, FALSE), (2, 'Gold Museum', 'Museum full of g
old pieces from ages ago', 1, 4.5, FALSE), (3, 'Comuna 13', 'Paisas making a hill something dang
erous like puntazons en el corazon', 2, 3, TRUE);
INSERT 0 3
lab=#
```

Now, we are gonna validate that the information is actually in the system. To do this, in the last section of this lab, we find the querys done by Dbeaver.

SQL SERVER – WINDOWS SERVER

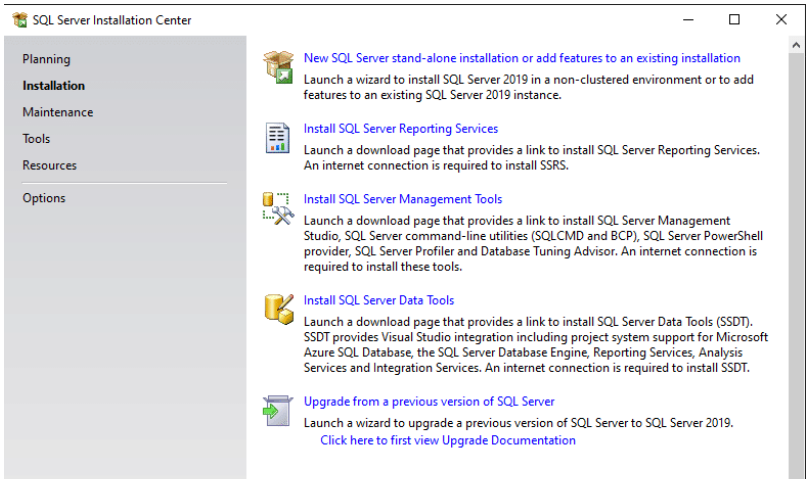
For this second database, we'll use the Windows Server machine with a graphical interface. Here, we'll use a web browser to download the DBMS software that will allow us to create our database. In this case, we'll use SQL Server, which was obtained from the official Microsoft website (https://www.microsoft.com/es-co/sql-server/sql-server-downloads). Here, we'll download the `***.exe***` file and then begin the installation of the Express version, wich allows us to modify some parameters that could be nescesary to conects external tools.

To begin the installation, we'll run the SQL Server installer and select custom mode. In this mode, we'll have to wait a while after entering the installation path while the program loads.

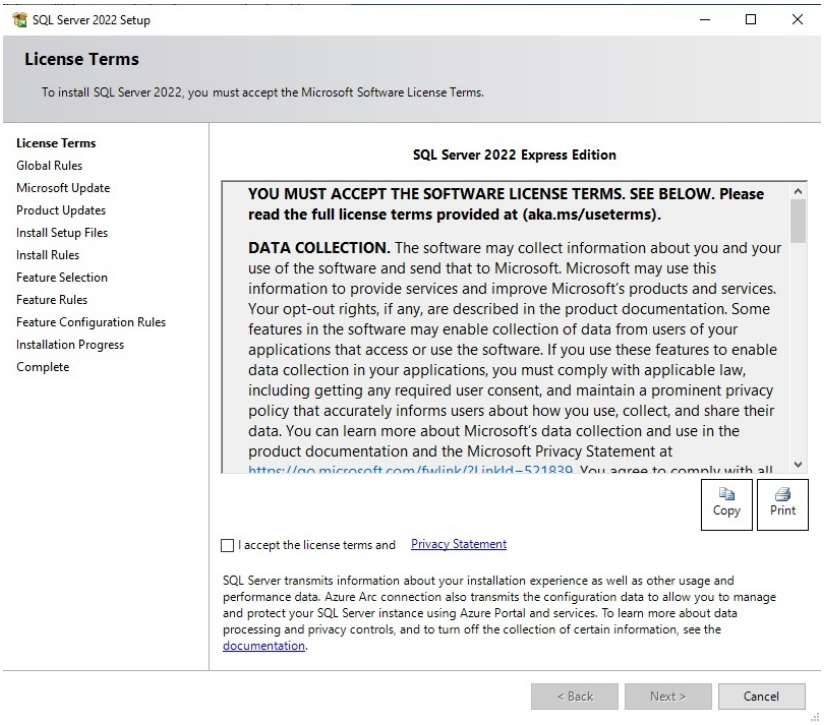


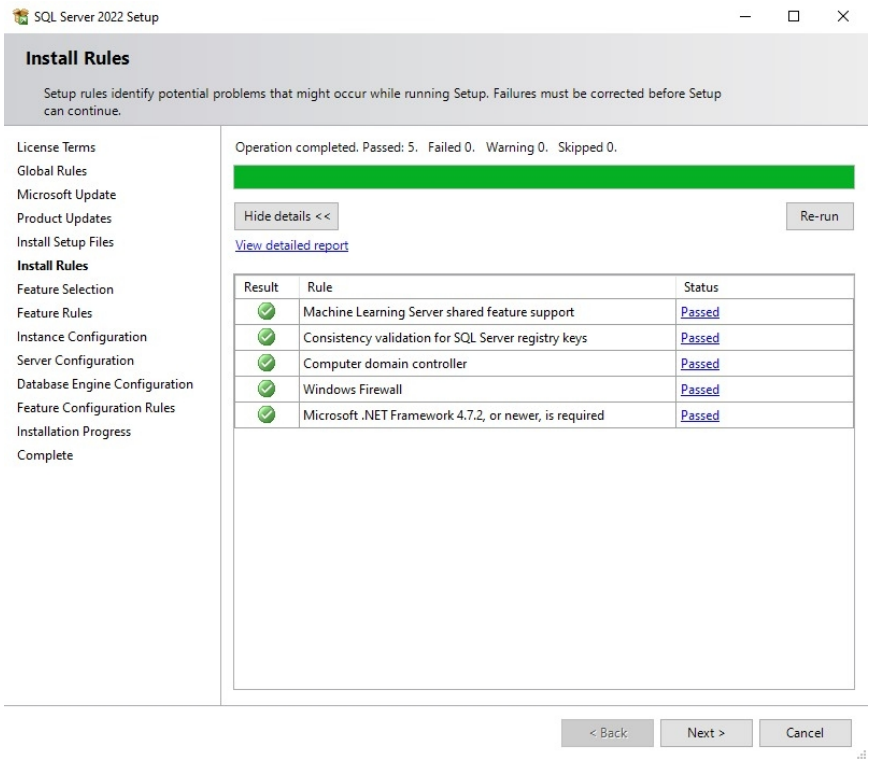
After installation, a screen with multiple options appears. Here, we can configure our data server by selecting the options we need. This mean, we'll perform all the necessary configuration to create our database.

When the program is created and executed correctly, it will display a screen with all the options for installing tools, drivers, and other SQL Server-related software. In our case, we're only interested in two options. Option 3 will allow us to download the manager, and option 1 will download the database engine server along with it.

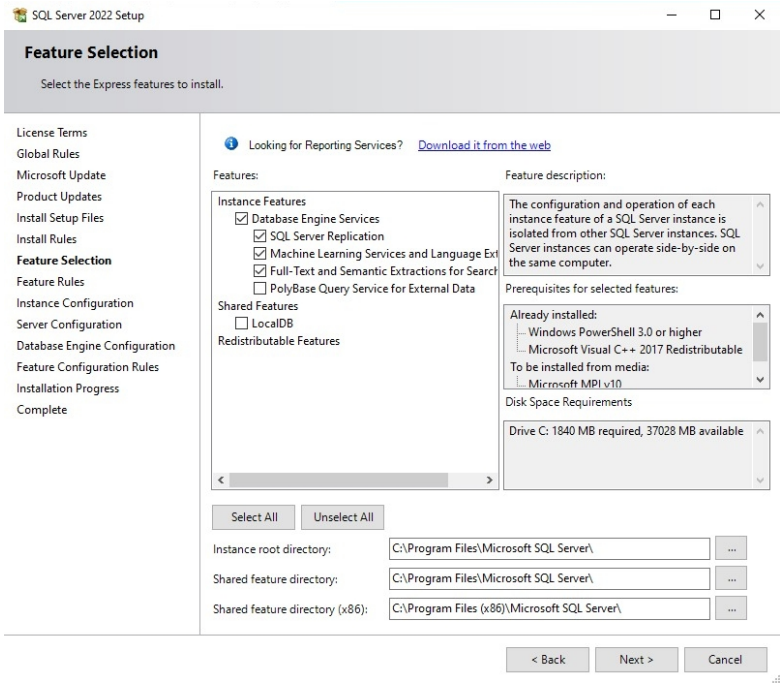


First, we'll select the first option. Here, we'll follow the step-by-step instructions to achieve the configuration we need for our database. Next, we'll see each screen that appears, and it will only clarify what we selected and why for each one. This way, we ensure a completely transparent installation and can be registered if we require a new machine.

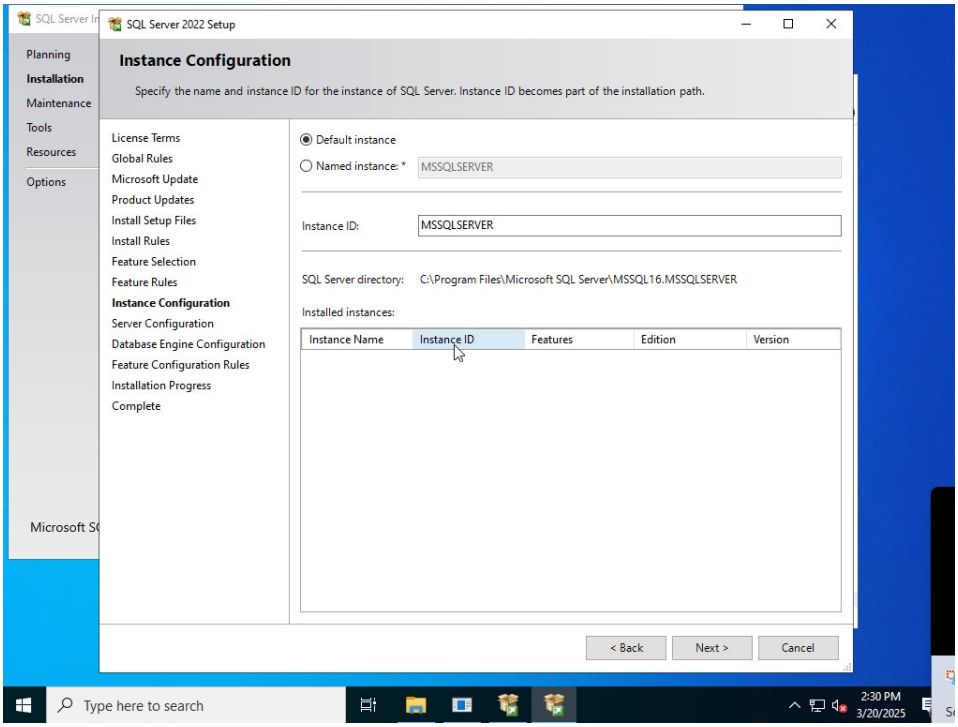


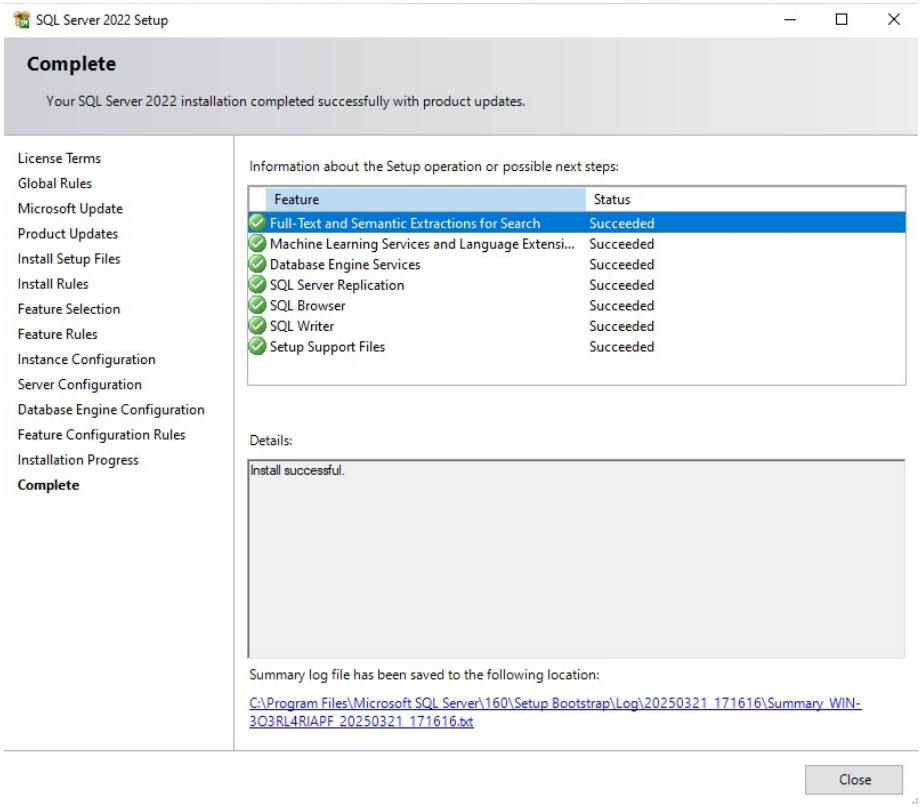


On the next screen, we'll leave the default options to avoid potential conflicts that may occur in the database.

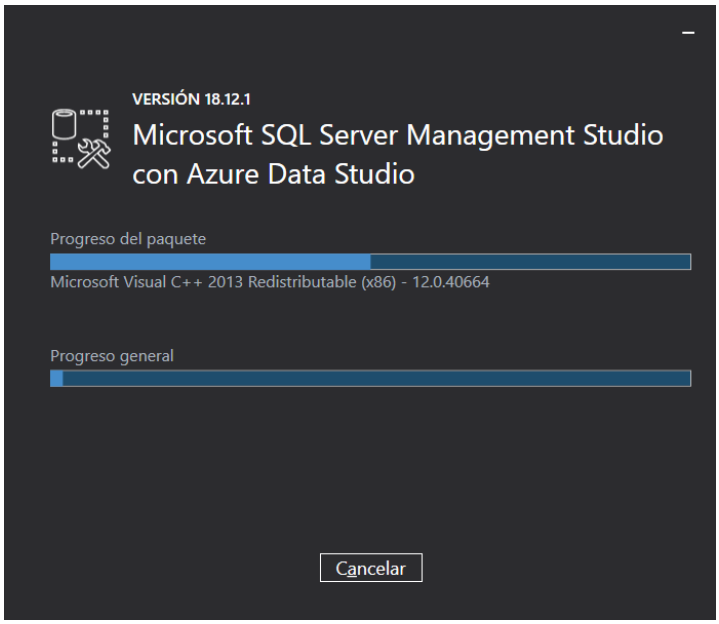


We select the default screen, as this assigns us the intrinsic user and password in the computer user, so we do not need to enter passwords or superusers.

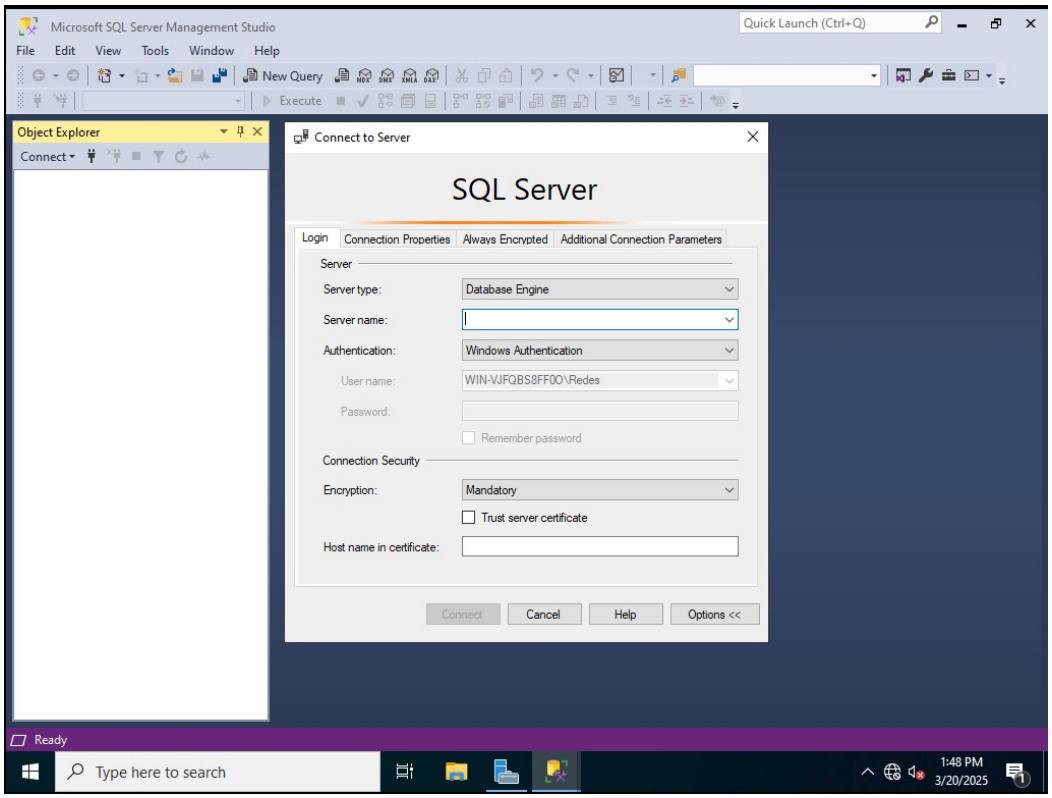




Finally, let's download the software we'll use to manage the database we're going to create. That is, we'll get SQL Server Management Studio (SSMS). In it, we can create tables, edit them, create users, run queries, enter data, delete data, etc., just like with any other database management system. We got it from the official page of Microsoft (https://learn.microsoft.com/es-es/ssms/download-sql-server-management-studio-ssms?view=sql-server-ver15).

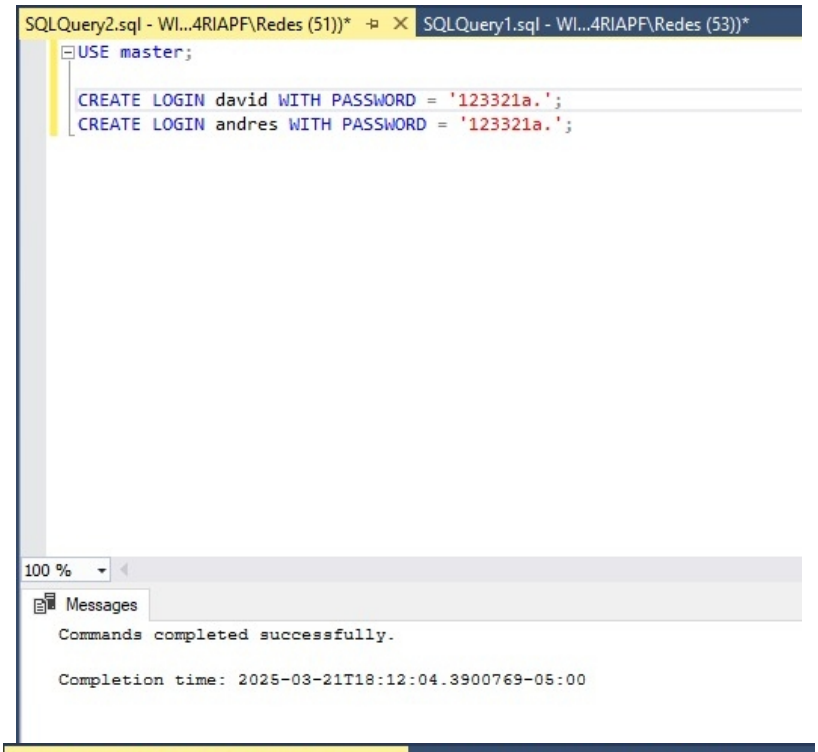


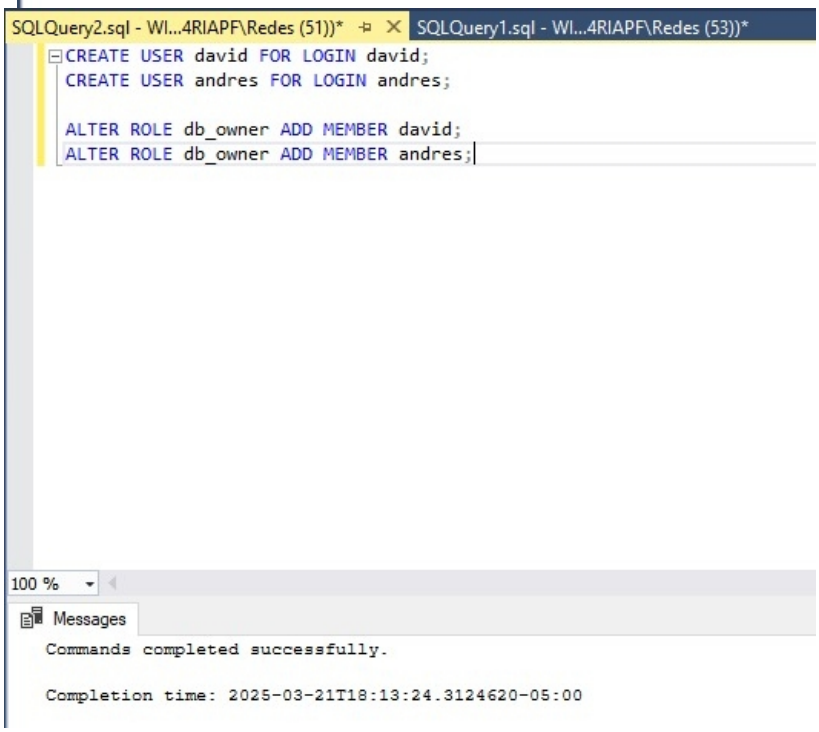
With all the required software downloaded, we can begin using these tools to create the database. To do so, we start on our local database server, logging in with the aforementioned credentials. This allows us to interact with the software and tools, such as scripts. These will be used to generate all the tables, relationships, etc.



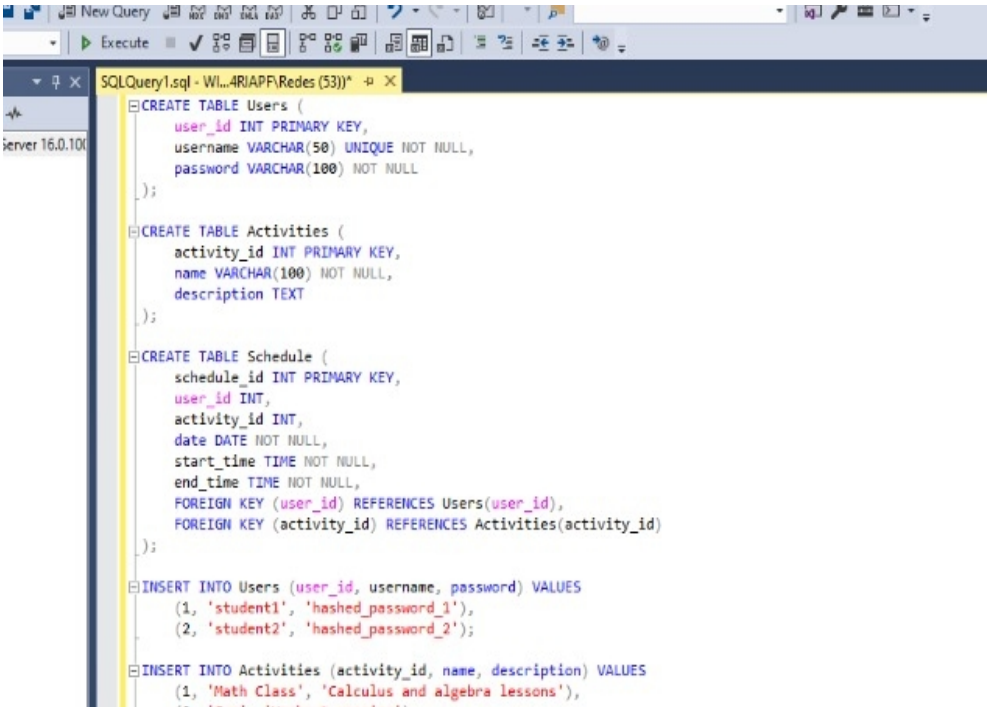
In this case, we were asked to create a database to organize monthly tasks or activities. We also needed to create a user for each group member. Therefore, we created the following tables, usernames, and permissions.

- Users:





- Tables:

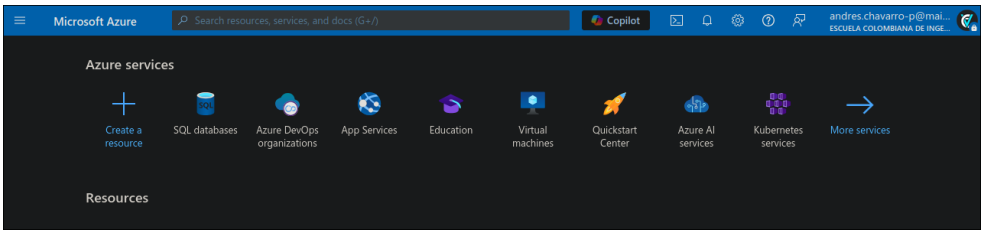


- Data insert:

Now, we are gonna validate that the information is actually in the system. To do this, in the last section of this lab, we find the querys done by Dbeaver.

SQL DATABASE – AZURE

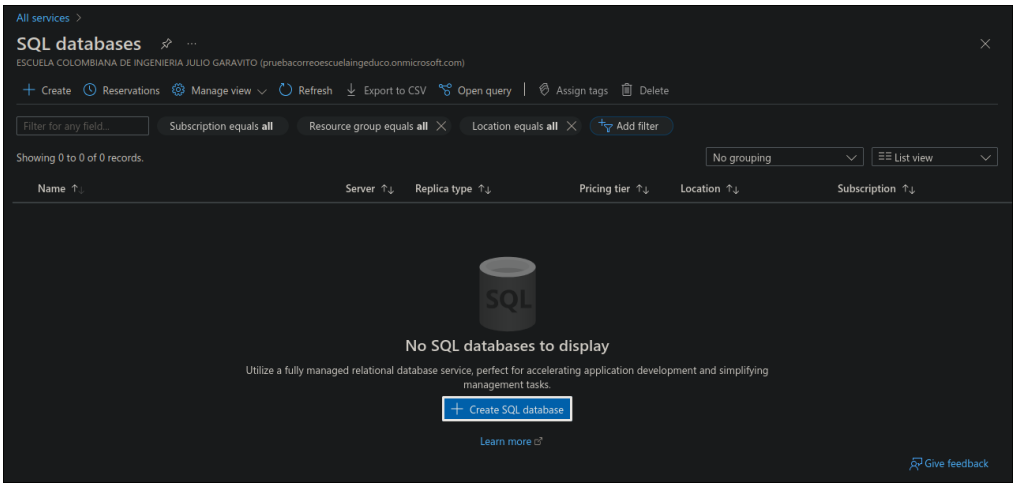
If we want to deploy a database to a cloud service, in this case Azure, we need an account. In our case, we had already created one, so we skipped that step. Likewise, it's not complicated and only requires registering with your institutional email.



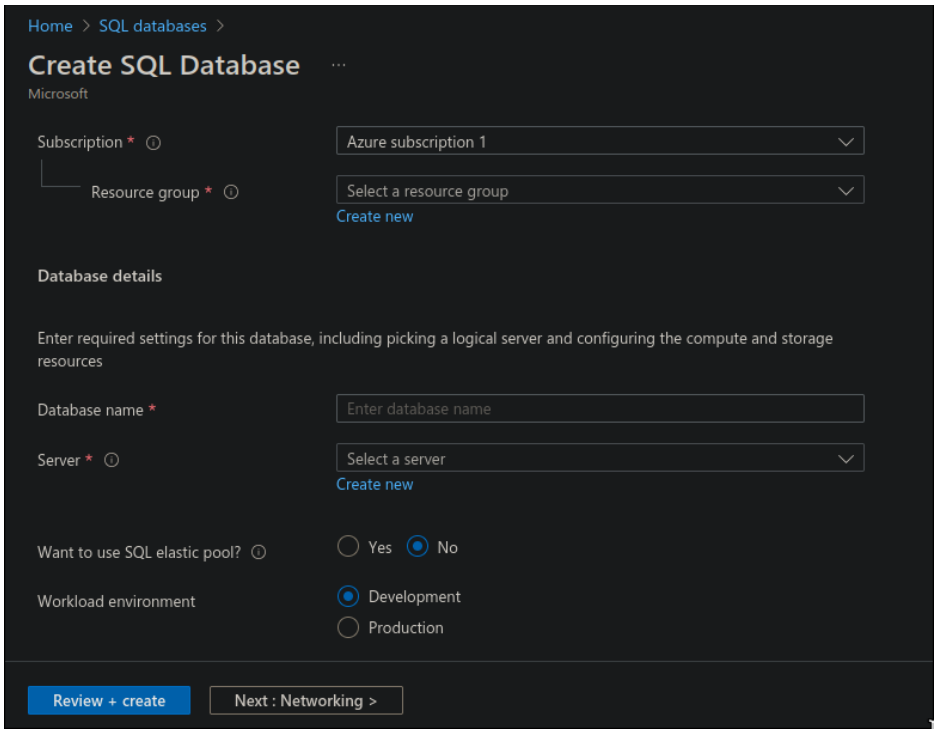
Before we continue, we need to understand how Azure works and what it's made up of. This will help us draw on the theoretical framework to make the following observations:

- Cloud computing allows access to computing resources over the internet, providing flexibility and scalability. Microsoft Azure reduces operational costs, improves resource availability, and enables quick scaling compared to maintaining on-premise infrastructure.
- Microsoft Azure offers three main service models: IaaS provides virtualized infrastructure like servers and storage, PaaS offers a platform for developing and deploying applications, and SaaS delivers ready-to-use software applications.
- Azure's regions and availability zones ensure high availability and fault tolerance by distributing services across different physical locations. This minimizes downtime and ensures consistent performance even during hardware or network failures.
- Vertical scaling increases the resources of a single server, which is suitable for improving the performance of individual workloads. Horizontal scaling adds more servers to distribute the load, making it ideal for handling high traffic or large datasets.
- TLS encrypts communication between the client and the server, securing data transfer and preventing unauthorized access. This is crucial for Azure SQL Database since it operates over the internet, unlike local databases that might rely on internal network security.
- Local SQL databases rely on direct TCP connections within a private network, while Azure SQL Database handles TCP connections over the internet, requiring secure protocols like TLS for data integrity and security.

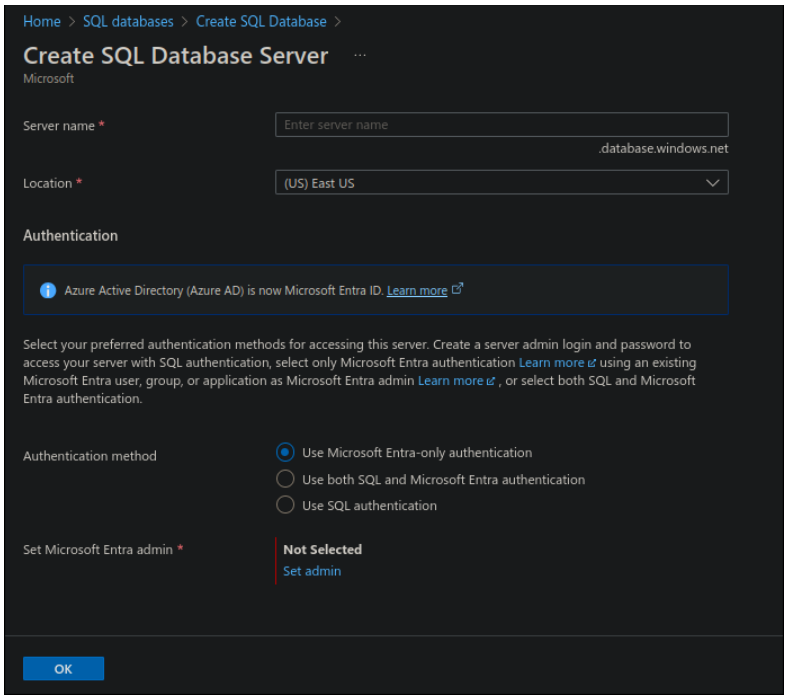
After understanding some Azure concepts, let's begin creating the database. To do this, we'll go to the "SQL Database" service. There, we'll find a screen that allows us to manage our databases. In our case, we'll create a new one.



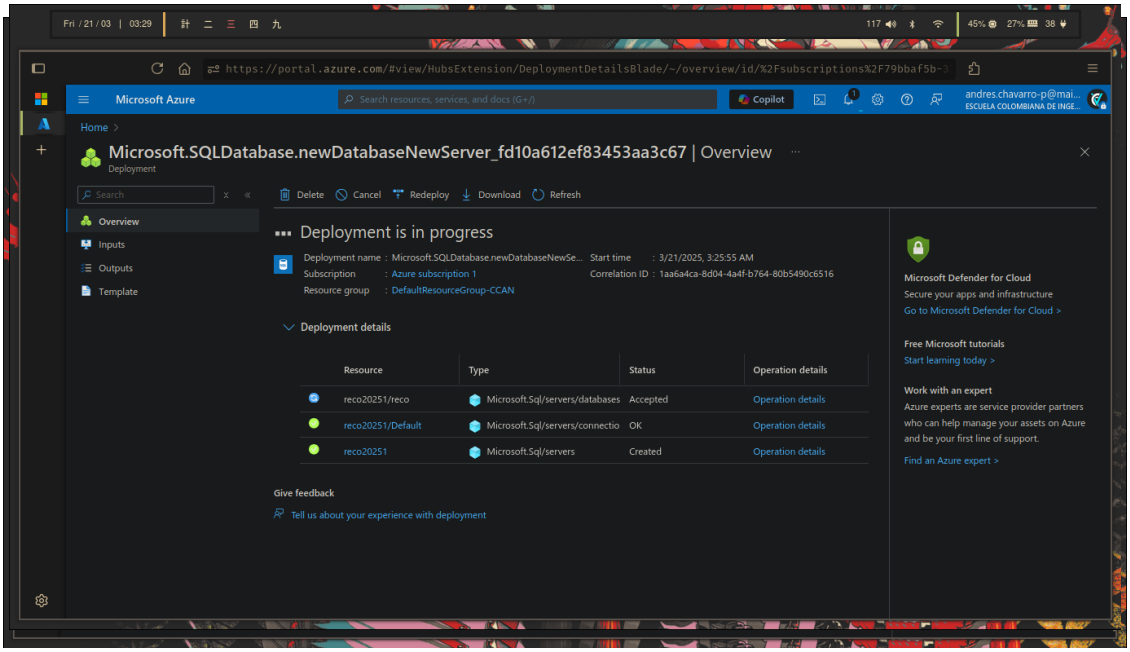
When trying to create a database, we have to configure a series of settings, such as name, server, type, and more. These settings are explained in more detail in the video requested for this section. However, these are more standard settings and don't require any additional configuration.



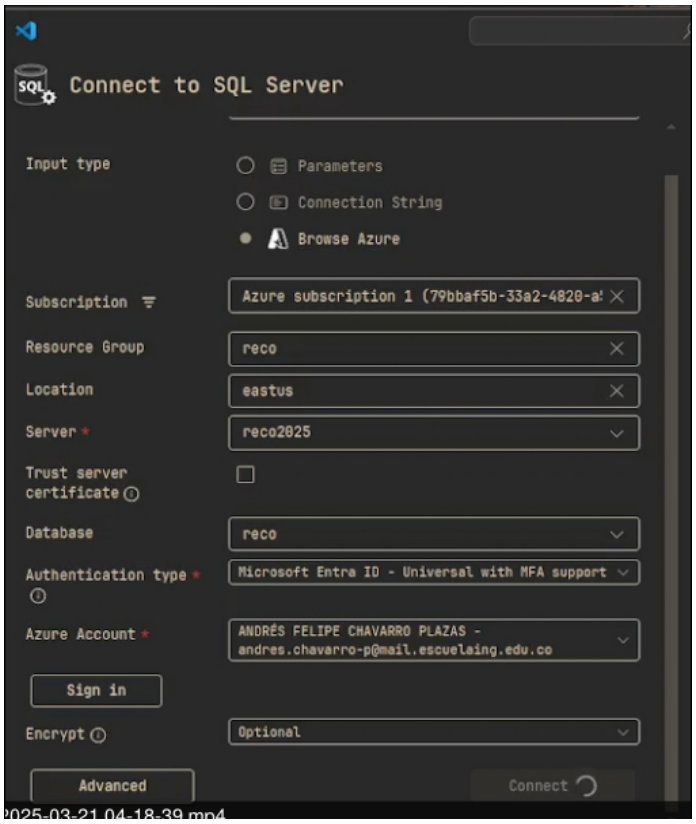
The only configuration we need to create is our own server installation. To do this, we are given a form that, when filled out, will create our portion of the infrastructure for deploying the database.



Once everything is created, we'll confirm that all the data is valid and proceed with creating the database in the cloud. While this is happening, we'll be notified that the database is being deployed.



Now, let's go to VSCode to connect to the database using the official Azure extension. To do this, use the Azure login and select the server where our different databases are hosted.



We begin by creating the tables and users, and inserting the necessary data to make the database fairly populated. In this case, we can run a script containing all the SQL code we need for this general task.

- Users:

```
1   ALTER ROLE db_owner ADD MEMBER andres;
2   ALTER ROLE db_owner ADD MEMBER david;
3   GO
4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN C

Messages

5:05:28 AM Started executing query at Line 62
Commands completed successfully.
Total execution time: 00:00:00.097

- Tables:

```
8   CREATE TABLE Authors (
1       author_id INT PRIMARY KEY,
2       author_name NVARCHAR(100),
3       affiliation NVARCHAR(100),
4       email NVARCHAR(100)
5   );
6
7   CREATE TABLE Books (
8       book_id INT PRIMARY KEY,
9       title NVARCHAR(255),
10      author_id INT FOREIGN KEY REFERENCES Authors(author_id),
11      publication_year INT,
12      isbn NVARCHAR(20),
13      publisher NVARCHAR(100),
14      edition NVARCHAR(50)
15  );
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE QUERY RESU

Messages

4:52:48 AM Started executing query at Line 8
Commands completed successfully.
Total execution time: 00:00:00.122

```sql
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title NVARCHAR(255),
    author_id INT FOREIGN KEY REFERENCES Authors(author_id),
    publication_year INT,
    isbn NVARCHAR(20),
    publisher NVARCHAR(100),
    edition NVARCHAR(50)
);

CREATE TABLE Articles (
    article_id INT PRIMARY KEY,
    title NVARCHAR(255),
    author_id INT FOREIGN KEY REFERENCES Authors(author_id),
    journal_name NVARCHAR(100),
    publication_year INT,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE QUERY RE

Messages

4:52:53 AM Started executing query at Line 15
Commands completed successfully.
Total execution time: 00:00:00.130

```sql
CREATE TABLE Articles (
    article_id INT PRIMARY KEY,
    title NVARCHAR(255),
    author_id INT FOREIGN KEY REFERENCES Authors(author_id),
    journal_name NVARCHAR(100),
    publication_year INT,
    volume NVARCHAR(20),
    issue NVARCHAR(20),
    doi NVARCHAR(100)
);

INSERT INTO Authors (author_id, author_name, affiliation, email)
VALUES
    (1, 'John Doe', 'University of Example', 'john.doe@example.com'),
    (2, 'Jane Smith', 'Research Institute', 'jane.smith@example.com');
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE QUERY RESULTS (PREVIE

Messages

4:52:58 AM Started executing query at Line 25
Commands completed successfully.
Total execution time: 00:00:00.102

- Data insert:

```
36  INSERT INTO Authors (author_id, author_name, affiliation, email)
1   VALUES
2       (1, 'John Doe', 'University of Example', 'john.doe@example.com'),
3       (2, 'Jane Smith', 'Research Institute', 'jane.smith@example.com');
4
5   INSERT INTO Books (book_id, title, author_id, publication_year, isbn, p
6   VALUES
7       (1, 'Introduction to Database Systems', 1, 2020, '978-0131873254',
8       (2, 'Data Science for Beginners', 2, 2021, '978-1718500452', 'O''Re
9
10  INSERT INTO Articles (article_id, title, author_id, journal_name, publi
11  VALUES
12      (1, 'Machine Learning Techniques', 1, 'Journal of AI Research', 202
13      (2, 'Quantum Computing Applications', 2, 'Quantum Information Proce
14
15  USE master;
16  GO
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE QUERY RESULTS (PREVIE

Messages

```
4:53:05 AM    Started executing query at Line 36
              (2 rows affected)
              Total execution time: 00:00:00.103
```

```
41  INSERT INTO Books (book_id, title, author_id, publication_year, isbn, publisher, edition)
1   VALUES
2       (1, 'Introduction to Database Systems', 1, 2020, '978-0131873254', 'Pearson', '5th'),
3       (2, 'Data Science for Beginners', 2, 2021, '978-1718500452', 'O''Reilly Media', '1st');
4
5   INSERT INTO Articles (article_id, title, author_id, journal_name, publication_year, volume, i
6   VALUES
7       (1, 'Machine Learning Techniques', 1, 'Journal of AI Research', 2022, '12', '3', '10.1016
8       (2, 'Quantum Computing Applications', 2, 'Quantum Information Processing', 2023, '8', '1'
9
10  USE master;
11  GO
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE QUERY RESULTS (PREVIEW) COMMENTS

Messages

```
4:53:12 AM    Started executing query at Line 41
              (2 rows affected)
              Total execution time: 00:00:00.104
```

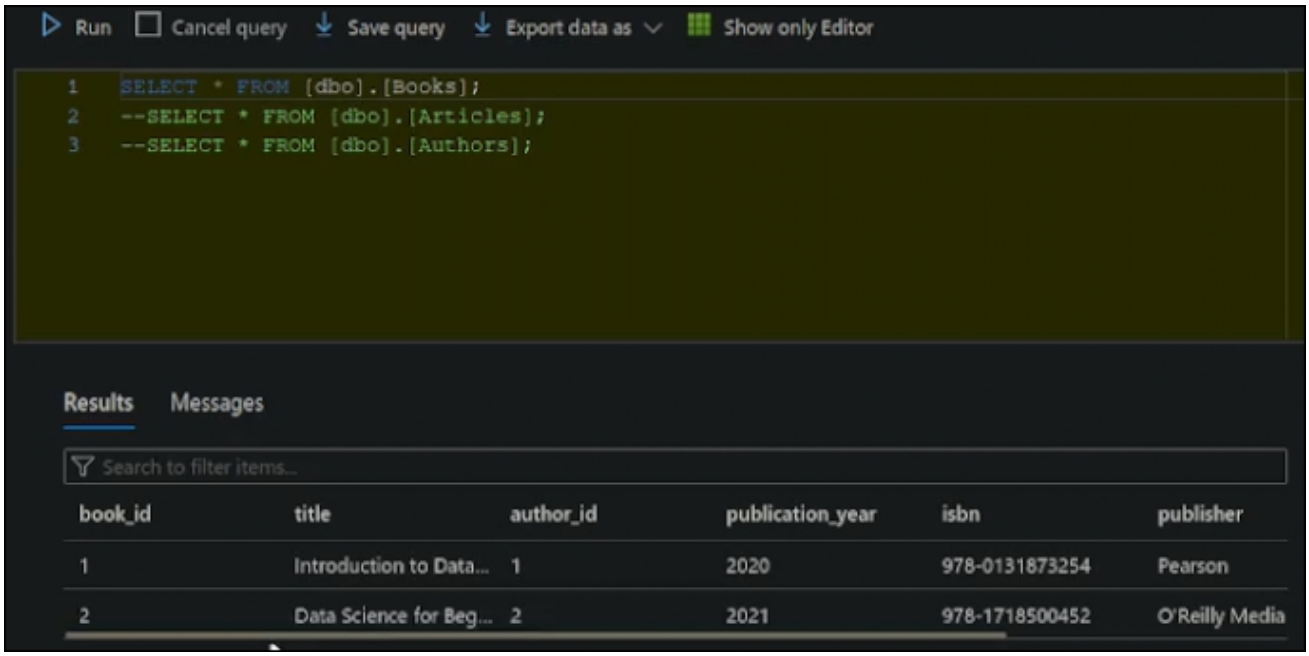
```
46  INSERT INTO Articles (article_id, title, author_id, journal_name, publication_year, volume, issue, doi)
1   VALUES
2       (1, 'Machine Learning Techniques', 1, 'Journal of AI Research', 2022, '12', '3', '10.1016/j.jair.2022.01.001'),
3       (2, 'Quantum Computing Applications', 2, 'Quantum Information Processing', 2023, '8', '1', '10.1007/s11128-023-039
4
5   USE master;
6   GO
7   CREATE LOGIN andres WITH PASSWORD = 'Reco123.';
8   CREATE LOGIN david WITH PASSWORD = 'Reco123.';
9   GO
10
11  USE reco;
12  GO
13  CREATE USER user1 FOR LOGIN andres;
14  CREATE USER user2 FOR LOGIN david;
```

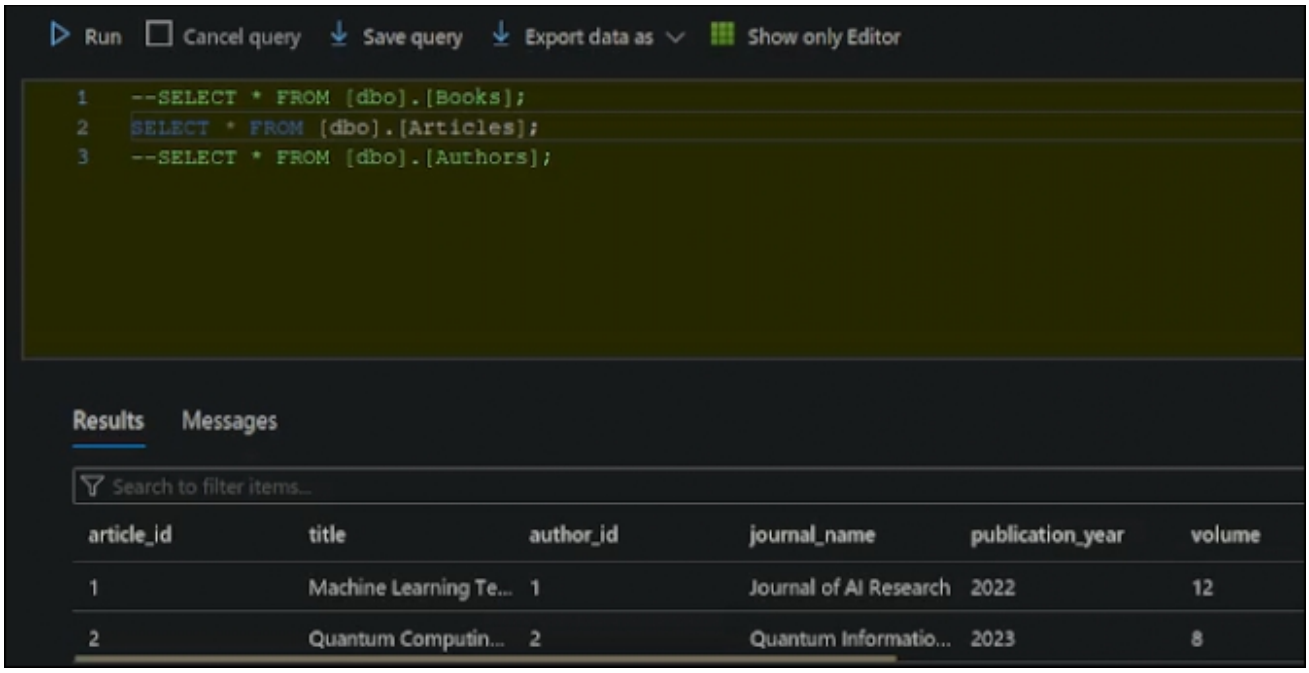
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE QUERY RESULTS (PREVIEW) COMMENTS

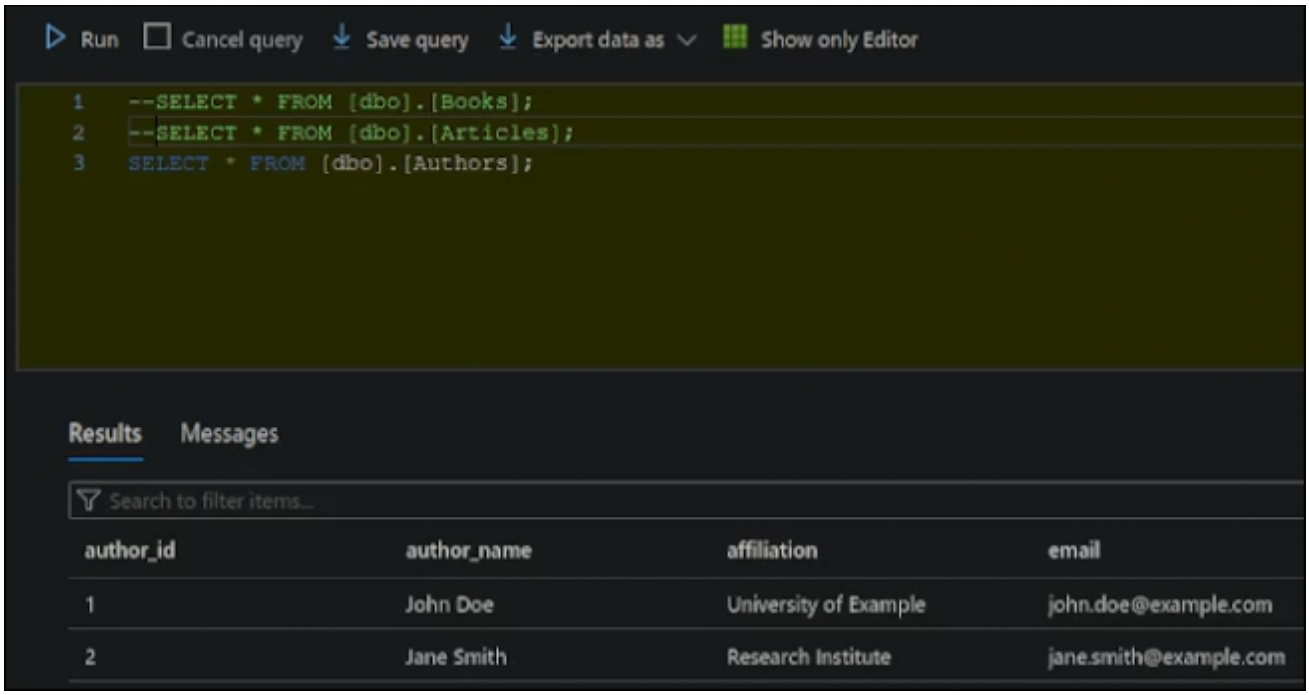
Messages Open in New Tab

```
4:53:26 AM    Started executing query at Line 46
              (2 rows affected)
              Total execution time: 00:00:00.102
```

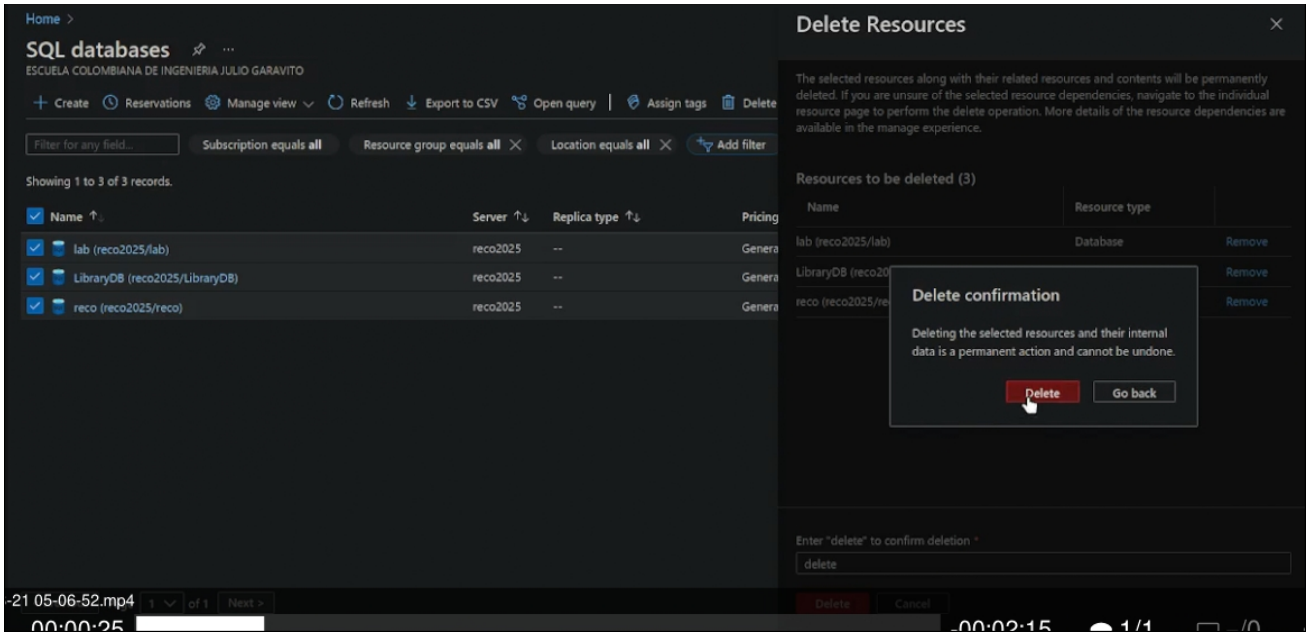
At this point, let's verify that the data was actually added to the database deployed in Azure. Therefore, we need to get into the Azure portal and access our instance. Next, we'll access the query area and see the tables that currently exist.







Finally, we need to delete the Azure database, as it has a cost that deducts from our available student credits. Then, we head to the SQL Server to delete it.



OTHER DB ENGINE CONFIGURATIONS

In this final section, we'll perform additional configurations on each operating system we use in this lab. These are intended to validate the correct operation of our databases, and we'll also use a remote viewer of these databases from a third-party system.

Database services generally don't start completely at system startup, as this means they consume resources and place an extra load on the operating system. For this reason, we'll initialize these services as soon as the machines we're working on start up, in this case Slackware.

To begin, in our case, Postgres starts using the `/usr/local/pgsql/bin/pg_ctl -D /usr/local/pgsql/data start` command from the user created to run the database. However, we'll automate this by adding a script to the folder designated for the `/etc/rc.d/` package starters. In this case, we'll create the `/etc/rc.d/rc.postgresql` file in which we'll place the aforementioned command. However, we decided to add several options directly to better manage the service.

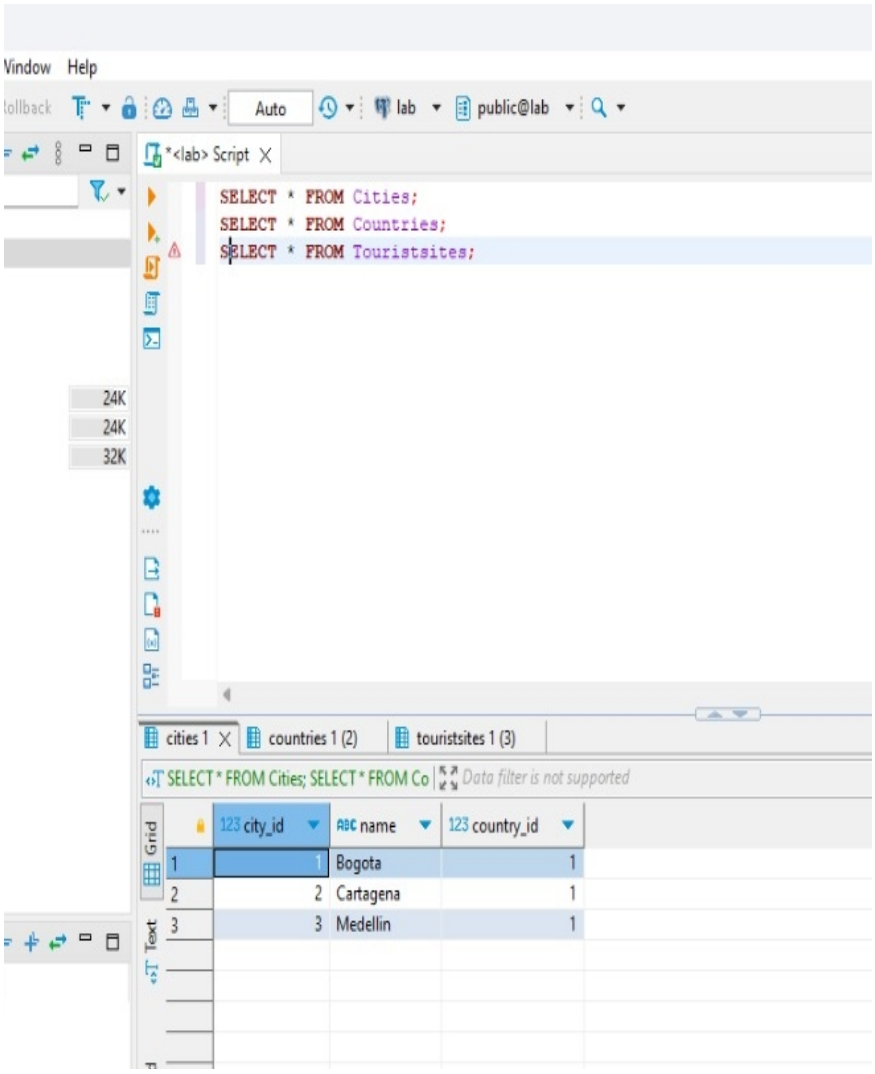
```
#!/bin/sh

PG_CTL="/usr/local/pgsql/bin/pg_ctl"
PG_DATA="/usr/local/pgsql/data"

case "$1" in
    start)
        echo "Iniciando PostgreSQL..."
        $PG_CTL -D $PG_DATA start
        ;;
    stop)
        echo "Deteniendo PostgreSQL..."
        $PG_CTL -D $PG_DATA stop
        ;;
    restart)
        echo "Reiniciando PostgreSQL..."
        $PG_CTL -D $PG_DATA restart
        ;;
    *)
        echo "Uso: $0 {start|stop|restart}"
        exit 1
        ;;
esac
```

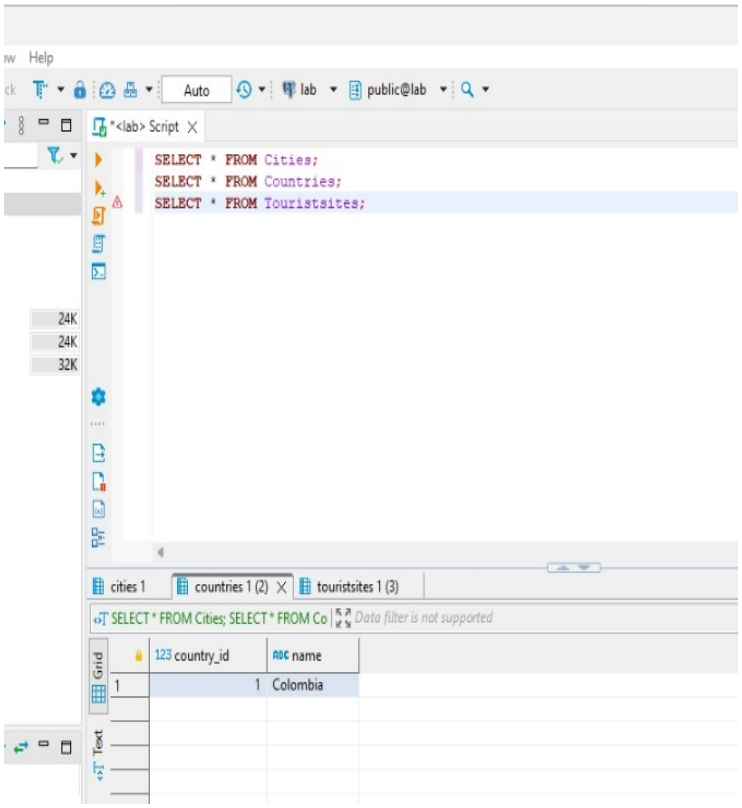
```
/etc/rc.d/rc.postgresql start

exit 0
```

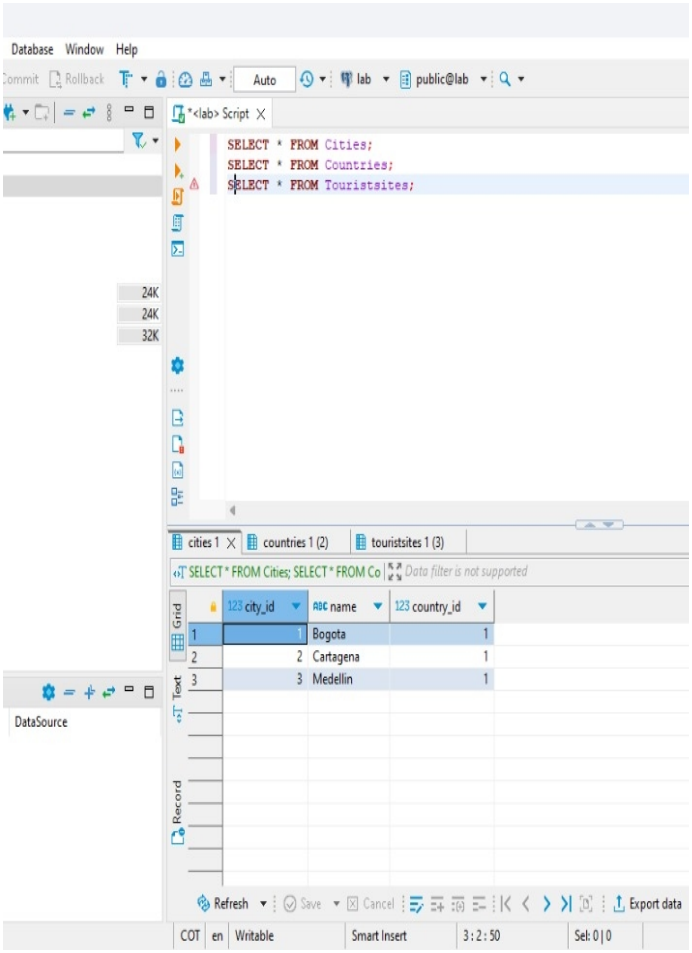
On the other hand, we will use the Dbeaver tool to validate and query the databases we created throughout this lab, except for Azure, which was requested to be removed so as not to waste platform resources.

When you start Dbeaver, there's an option to connect to our database. In this section, we'll set the IP number assigned to the machine that's running and has the database active as the host, and we'll enter the username and password of one of the previously created users. This way, we'll have access to the information we added and can validate that it's correct.

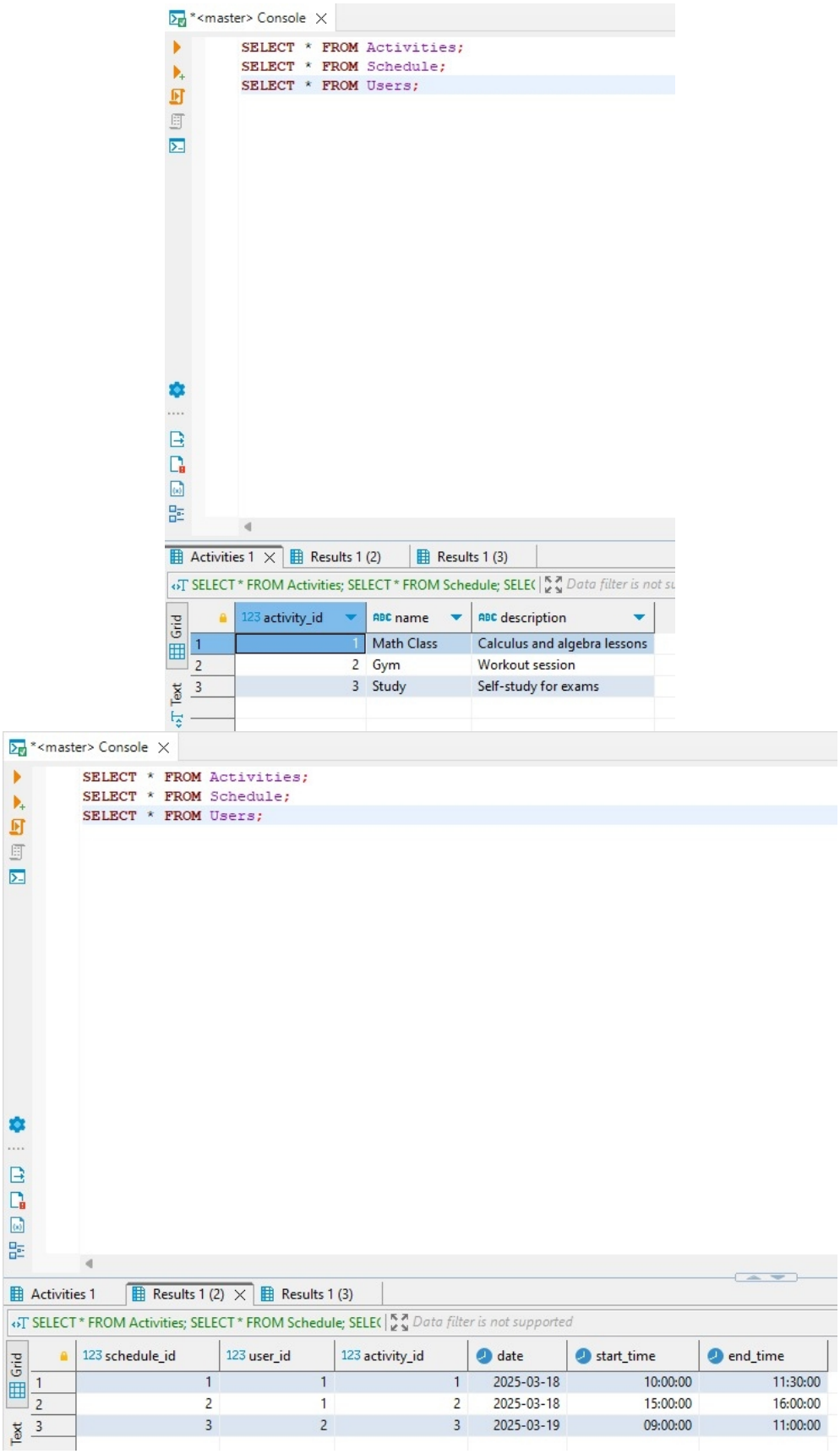
- PostgreSQL (Slackware):

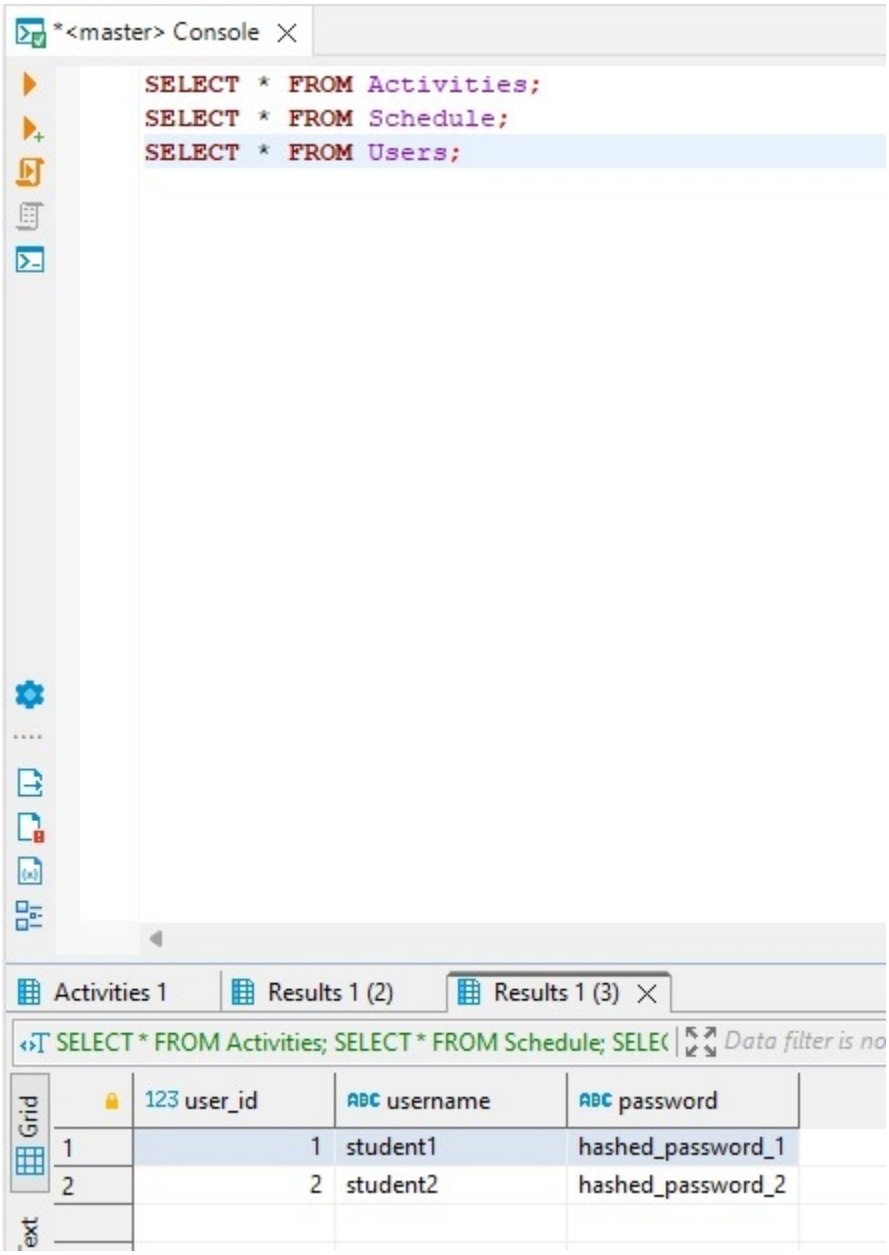



Help
Auto
lab
public@lab
*<lab> Script
SELECT * FROM Cities;
SELECT * FROM Countries;
SELECT * FROM Touristsites;
24K
24K
32K
cities 1
countries 1 (2)
touristsites 1 (3)
SELECT * FROM Cities; SELECT * FROM Co
Data filter is not supported
Grid
123 country_id
ABC name
1
1
Colombia
Text


Database
Window
Help
Commit
Rollback
Auto
lab
public@lab
*<lab> Script
SELECT * FROM Cities;
SELECT * FROM Countries;
SELECT * FROM Touristsites;
24K
24K
32K
cities 1
countries 1 (2)
touristsites 1 (3)
SELECT * FROM Cities; SELECT * FROM Co
Data filter is not supported
Grid
123 city_id
ABC name
123 country_id
1
1
Bogota
1
2
2
Cartagena
1
3
3
Medellin
1
Text
Record
DataSource
Refresh
Save
Cancel
Export data
COT
en
Writable
Smart Insert
3 : 2 : 50
Sel: 0 | 0

- SQL Server (Windows Servers):




*<master> Console
SELECT * FROM Activities;
SELECT * FROM Schedule;
SELECT * FROM Users;
Activities 1
Results 1 (2)
Results 1 (3)
SELECT * FROM Activities; SELECT * FROM Schedule; SELEC
Data filter is no
Grid
123 user_id
ABC username
ABC password
1 1 student1 hashed_password_1
2 2 student2 hashed_password_2

CONCLUTIONS

In this lab, we installed and configured several database management systems, including PostgreSQL on Slackware, SQL Server on Windows Server, and Azure SQL Database in the cloud. Each database was properly set up with specific users and customized permissions, and we inserted data to test functionality and ensure proper data storage and management. This helped us understand the differences between local and cloud-based databases and how permission and security settings affect data access and integrity. We also reinforced the importance of scalability, learning when to apply vertical or horizontal scaling to improve system performance and responsiveness.

Using Wireshark to analyze DNS, HTTP messages, and Ethernet frames allowed us to observe how network communication works and how domain names are resolved. This was helpful for identifying potential network issues and understanding the structure of data packets during transmission.

This lab was valuable for reinforcing key concepts about databases and networks while applying best practices to ensure the availability and security of configured services. It also helped us understand the importance of secure data transmission, especially in cloud-hosted databases, where protocols like TLS are essential for protecting information. The hands-on experience with different platforms and configurations enhanced our technical skills, which will be valuable for implementing and managing IT infrastructures in real-world environments.

VIDEOS

In this section you will find the videos made for this laboratory, these are:

- Summary and explanation of the deployment made in Azure.

  https://youtu.be/Eyb7CLNV9B0

BIBLIOGRAFY

17.3. Building and Installation with Autoconf and Make. (2025, febrero 20). PostgreSQL Documentation. https://www.postgresql.org/docs/current/install-make.html

About. (s/f). Postgresql.org. Recuperado el 23 de marzo de 2025, de https://www.postgresql.org/about/

Carson. (2021, mayo 4). *DNS message — how to read query and response message*. Medium. https://cabulous.medium.com/dns-message-how-to-read-query-and-response-message-cfebcb4fe817

Ethernet frame: definition and variants of the frame format. (s/f). IONOS Digital Guide. Recuperado el 23 de marzo de 2025, de https://www.ionos.com/digitalguide/server/know-how/ethernet-frame/

How to create portable Linux binaries. (s/f). Ycombinator.com. Recuperado el 23 de marzo de 2025, de https://news.ycombinator.com/item?id=34762779

HTTP messages. (s/f). MDN Web Docs. Recuperado el 23 de marzo de 2025, de https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages

Improve, G. (2017, octubre 3). *Ethernet frame format*. GeeksforGeeks. https://www.geeksforgeeks.org/ethernet-frame-format/

Miles, S. ‘geosmiles’. (2023, octubre 4). *DBeaver*. Medium. https://geosmiles.medium.com/dbeaver-93a16c52ad2c

Slingerland, C. (2024, julio 18). *Horizontal vs. Vertical scaling: Which should you choose?* CloudZero. https://www.cloudzero.com/blog/horizontal-vs-vertical-scaling/

Wikipedia contributors. (2025, marzo 21). *Cloud computing*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Cloud_computing&oldid=1281701262

WilliamDAssafMSFT. (s/f). *¿Qué es el servicio Azure SQL Database? - Azure SQL Database*. Microsoft.com. Recuperado el 23 de marzo de 2025, de https://learn.microsoft.com/es-es/azure/azure-sql/database/sql-database-paas-overview?view=azuresql

(S/f-a). Cloudflare.com. Recuperado el 23 de marzo de 2025, de https://www.cloudflare.com/es-es/learning/ssl/transport-layer-security-tls/

(S/f-b). Amazon.com. Recuperado el 23 de marzo de 2025, de https://aws.amazon.com/what-is-cloud-computing/